



The complete project guide

Continual-Learning Graph Neural Network Intrusion Detection System

Final-year B.Tech project · repository [Continuous-learning-IDS](#)

Everything about the project, explained from the beginning in plain language: the problem, the research behind it, the model, the data, every page and button of the console, how it was built, and what the results really say.

Prepared 6 October 2026. Every number in this guide is copied from a result file in the repository; nothing is estimated. Where a number comes from a single run, the guide says so.

Contents

1. The project in one page
2. Scope: what it is, what it is not, and who it is for
3. The real-world problem, with examples
4. The ideas you need, explained simply
5. Prior research: 40 papers, what each did, and what each leaves open
6. The gap: what was missing, and what this project does about it
7. The machine-learning model: why a graph network, and why not Random Forest
8. The data: why CIC-IDS2017 and CSE-CIC-IDS2018, why two, and why not newer ones
9. How the product works, end to end
- L0. Every page and every feature of the console, with screenshots
- L1. How it was built: stack, structure, tests, and the API key
- L2. What the results say (and do not say)
- L3. Limitations and future work
- L4. Things you might have missed: viva questions, demo plan, glossary, commands

1 · The project in one page

In one sentence: a network intrusion detection system that looks at traffic as a **graph of computers talking to each other**, keeps **learning new kinds of attack without forgetting the old ones**, notices by itself when traffic has **changed**, and turns thousands of alerts into a handful of **explained incidents** that a human approves or rejects.

The problem in plain words

Companies watch their network traffic for attacks. Most machine-learning detectors are trained once on a fixed dataset and then frozen. Three things go wrong in practice:

1. **New attacks appear.** A detector trained in January has never seen the attack that arrives in March.
2. **Retraining makes it forget.** If you train the model again on only the new attack, it forgets the old ones. This is called *catastrophic forgetting*, and it is dramatic: in this project a normally retrained graph model went from catching 100 % of the first attack type to 0 %.
3. **Analysts drown.** A per-flow detector raises one alert per network conversation. A single scan can produce 5,000 alerts in a few seconds. Studies of real security teams describe most of their alerts as false positives (Alahmadi et al., USENIX Security 2022).

What we built

Detector	An E-GraphSAGE graph neural network that classifies every network flow using both the flow's own numbers and what the two computers involved are doing with everyone else.
Learning over time	Continual learning: new attack types are learned one after another. Two tools stop forgetting: replay (keep a few old examples and rehearse them) and EWC (protect the parts of the model that mattered for old attacks).
Noticing change	An ADWIN drift detector watches the model's error rate and triggers retraining only when traffic really changes.
Usable output	Alerts are grouped into incidents , each with an explanation , an "I'm not sure" abstention , a novelty flag for unfamiliar traffic, and a proposed firewall rule that is never executed (dry run only).
Console	An eight-page web console (Overview, Incident queue, Live stream, Graph explorer, Classify, Models, Adaptation & trust, Reproducibility) served by a FastAPI backend.
Proof	Every number comes from scripts in experiments/, run on two public datasets, with multiple random seeds, a realistic time-ordered test split, statistics, 92 automated tests and a browser smoke test.

The headline results, honestly

Question	Answer	How sure
Does it learn new attacks without forgetting?	Yes. 0.915 macro-F1 on the time-ordered test split (0.964 on the easier interleaved split), first attack still caught 99.8–100 %. Normal retraining forgets it completely (0 %).	5 seeds and 3 seeds
Does the graph help on attacks it has seen?	A little: +0.036 to +0.044 macro-F1 over an identical per-flow model.	Indicative (bootstrap interval excludes zero, too few seeds for a significance test)
Does the graph help on attacks it has never seen?	Clearly but unevenly: on CSE-CIC-IDS2018 it caught more than 80 % of a never-seen attack in 6 of 12 tests; the per-flow model caught at most 0.07 % in all 12. Which attacks it catches changes between training runs, except DoS and DDoS.	2 seeds
Is replay what stops forgetting?	Yes. Replay alone goes from 0.304 (nothing stored) to 0.952 with five stored windows per attack type. EWC alone fails (0.300).	3 seeds
Does ADWIN retrain at the right time?	It finds the changes, but it is not cheaper than a fixed schedule. It is better than a schedule when two attacks arrive mixed together (0.968 vs 0.926).	3 seeds
Is it fast enough?	A 5,000-flow window is scored in 5.5 ms on a laptop GPU (about 628,000 flows per second).	Measured
Can analysts cope?	About 100,000 flagged flows became about 50 incidents, 84 % of them real attacks.	Single run

2 · Scope: what it is, what it is not, and who it is for

What it is

A **research prototype of a triage layer**: software that sits beside an organisation's existing intrusion detection system, reads the network flows, and helps a small security team decide what to look at first. It is built so that every claim can be checked: the experiments, the console and the documentation all read the same result files.

What is in scope

- Classifying network flows (benign, or one of seven attack categories) from 83 flow features.
- Learning attack categories one after another without forgetting (continual learning).
- Detecting when traffic changes and deciding when to retrain (drift detection).
- Detecting attack types the model was never trained on (unseen attacks, novelty).
- Reducing analyst workload: incidents, explanations, abstention, proposed (never executed) responses.
- An honest evaluation: two datasets, multiple seeds, a time-ordered split, statistics, failure cases.
- A working console and API that a person can use and demonstrate.

What is deliberately out of scope

- **Blocking traffic automatically.** Nothing ever executes a firewall rule; every action is a dry run.
- **Replacing a SOC platform.** There is no case management, no correlation across log sources, no TLS, no user roles. The README and the positioning note say so plainly.
- **Reading packet contents.** Only flow statistics (sizes, timings, flags) are used, never payloads.
- **Capturing traffic.** Flows arrive as CSV in the CICFlowMeter format; a production system would ingest Zeek or Suricata output directly.
- **Production validation.** All results come from two public laboratory datasets.

Target audience

Who	What they get from it
Primary user: a small security team (2–5 analysts) that already runs a signature IDS such as Suricata, Snort or Zeek and has more alerts than it can read.	Incidents instead of raw alerts, a reason for each one, an "unsure" bucket, a flag for unfamiliar traffic, and a model that keeps up with new attacks.
Examiners and reviewers	A system whose every number can be regenerated with one command, with the negative results reported next to the positive ones.

Who	What they get from it
Students and researchers working on graph-based or continual intrusion detection	A complete, tested pipeline (data → graphs → continual training → drift → evaluation → API → UI) to build on.

Positioning in one line (from docs/positioning.md): *a continual-learning triage layer on an existing IDS's alert stream, not a SOC platform.*

3 · The real-world problem, with examples

Example 1: the port scan that looks innocent one connection at a time

An attacker's computer (172.16.0.1) tries to connect to 996 different ports on one server in seven seconds. Each single connection is tiny and looks like a normal failed request. A detector that judges **one flow at a time** sees 4,986 boring flows. A detector that looks at the **graph** sees one computer reaching out to almost a thousand ports on the same victim: an unmistakable scan. This exact case appears in the Incident queue screenshot in section 10.

Example 2: the detector that forgets

Monday the model learns BruteForce (password guessing). Wednesday it is retrained on DoS (denial of service). On Thursday BruteForce comes back, and the retrained model no longer recognises it. In this project's measurements a normally retrained graph model catches **0 %** of the first attack after learning the later ones, while the continual model still catches **99.8–100 %**.

Example 3: the attack nobody trained on

A new botnet appears. No labelled examples exist yet. A per-flow model has never seen anything like it and calls it benign. A graph model can still notice that one host is talking to many others in an unusual pattern. On CSE-CIC-IDS2018 the per-flow model detected at most 0.07 % of any never-seen attack; the graph model detected more than 80 % in half of the tests.

Example 4: 100,000 alerts

On the CIC-IDS2017 test traffic, the model flagged about 100,000 flows. Grouping flows that belong to the same attacker and victim turned that into about 50 incidents, 84 % of them real attacks. An analyst can read 50 incidents in a shift; nobody can read 100,000 alerts.

Example 5: traffic changes

Software updates, new users and new attacks change what "normal" looks like. Retraining every day wastes effort; never retraining lets accuracy rot. In this project a model that never adapts falls to 0.230 macro-F1 on the stream, while adapting keeps it around 0.96.

4 · The ideas you need, explained simply

Network flow

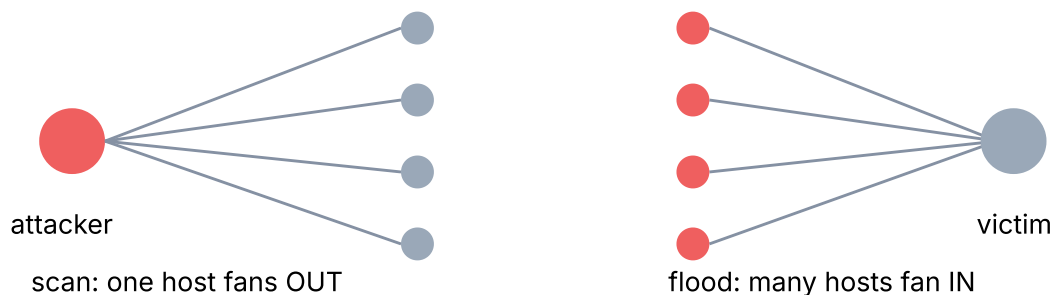
One conversation between two computers, summarised as numbers: how long it lasted, how many packets went each way, how many bytes, which TCP flags were set, the time between packets, and so on. CICFlowMeter turns raw traffic into these rows. This project uses **83 numeric features** per flow. IP addresses are **never** used as features; they only decide who is connected to whom in the graph.

Intrusion detection system (IDS)

Software that watches traffic for attacks. **Signature** IDSs (Snort, Suricata) match known patterns: precise, but blind to anything new. **Machine-learning** IDSs learn patterns from labelled examples: they can generalise, but they are usually trained once and frozen.

Graph

Dots and lines. Here every **computer (IP address) is a dot (node)** and every **flow is a line (edge)** between two dots. Each graph holds **5,000 consecutive flows**, called a *window*.



Attacks have shapes. A per-flow model sees only one line at a time; a graph model sees the shape.

Graph neural network (GNN), E-GraphSAGE

A neural network that works on graphs. For each flow it looks at the flow's own numbers, then **gathers summaries of the neighbouring flows** (other flows of the same two computers), and combines both to decide. GraphSAGE (Hamilton et al., 2017) introduced this "sample and aggregate" idea; **E-GraphSAGE** (Lo et al., 2022) adapted it so that the **edges (flows)** carry the features and get classified. Our model has 69,128 learnable numbers (parameters) and takes milliseconds per window.

Continual learning and catastrophic forgetting

Learning tasks one after another, like a student taking one subject per term. A normal neural network that studies the new subject overwrites what it knew about the old one: **catastrophic forgetting**. Two cures are used here:

- **Replay** (Rolnick et al., 2019): keep a small box of old flashcards and mix a few into every study session. Here the box holds up to 10 old traffic windows per attack type, and two are rehearsed at

each training step.

- **EWC, Elastic Weight Consolidation** (Kirkpatrick et al., 2017): measure which knobs inside the model mattered most for the old subject, and make those knobs stiff, so learning the new subject moves them as little as possible.

Concept drift and ADWIN

Drift means the traffic the model sees has changed since it was trained. **ADWIN** (Bifet & Gavalda, 2007) keeps a window of recent error values and constantly asks: "is the older half of this window significantly different from the newer half?" If yes, it raises a drift flag and the model retrain on the most recent traffic. It is a smoke alarm for accuracy.

Metrics

- **Macro-F1**: detection quality averaged over every category, so a rare attack counts as much as a common one. 1.0 is perfect.
- **Retention**: how much of the first attack type is still caught after learning all the others. 1.0 = nothing forgotten.
- **False-positive rate (FPR)**: share of normal traffic wrongly flagged. 0.07 % means 7 false alarms per 10,000 normal flows.
- **BWT (backward transfer)**: how much older tasks got worse after later training; near 0 is good, negative is forgetting.

Conformal prediction (abstaining)

A way to let the model say "I'm not sure". For each class it learns, on validation data, how low the model's confidence can be for genuine members of that class. If a new flow fits more than one class, or none, the model **abstains** and hands it to an analyst (Angelopoulos & Bates, 2021).

Calibration

Whether "90 % confident" really means "right 90 % of the time" (Guo et al., 2017). Measured with expected calibration error (ECE).

Open-set / novelty detection

Scoring how unfamiliar a flow looks, so that never-seen attacks can be flagged as "new" rather than forced into a known class. Three scores are compared: maximum softmax probability (Hendrycks & Gimpel), energy (Liu et al., 2020) and distance to class prototypes.

5 · Prior research: 40 papers

Every paper below was checked against its publisher or arXiv record (title, authors, year). **35 of the 40** are open access and were downloaded into the repository folder `research paper/`; the other five are behind a paywall or a bot check and are listed with their link. "What it leaves open" is relative to *this project's* goal, not a criticism of the paper. File names start with the number in the first column.

5.1 Datasets and how to evaluate fairly

#	Paper	What it did	What it leaves open for us
01	Sharafaldin, Lashkari, Ghorbani. <i>Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization</i> . ICISSP 2018.	Created CIC-IDS2017: five days of realistic office traffic with scheduled attacks, converted to flows with CICFlowMeter.	The released labels and flows contain errors; static use only.
02	Engelen, Rimmer, Joosen. <i>Troubleshooting an Intrusion Detection Dataset: the CICIDS2017 Case Study</i> . IEEE S&P Workshops 2021.	Found flow-construction, feature and labelling problems; reconstructed or relabelled more than 20 % of the traffic.	A corrected dataset, but no continual or graph evaluation. We use this corrected release.
03	Liu, Engelen, Lynar, Essam, Joosen. <i>Error Prevalence in NIDS Datasets: A Case Study on CIC-IDS-2017 and CSE-CIC-IDS-2018</i> . IEEE CNS 2022. (paywalled ; record: archive.cps-vo.org/node/97449)	Extended the error analysis to CSE-CIC-IDS2018 and published a corrected version.	Same: data quality, not detection. We use this corrected release.
04	Lanvin et al. <i>Errors in the CICIDS2017 Dataset and the Significant Differences in Detection Performances It Makes</i> . CRiSIS 2022. (hal.science/hal-03775466 , bot check)	Showed that label errors change reported detection results substantially.	Confirms why corrected data matter; no new detector.
05	Flood, Engelen, Aspinall, Desmet. <i>Bad Design Smells in Benchmark NIDS Datasets</i> . IEEE EuroS&P 2024 (Distinguished Paper). (IEEE Xplore 10629000)	Analysed seven benchmark datasets and named six recurring design flaws, such as low diversity and murky labels.	Warns that lab benchmarks may not generalise: a limitation we state for our own results.
06	Arp et al. <i>Dos and Don'ts of Machine Learning in Computer Security</i> . USENIX Security 2022.	Catalogued ten common pitfalls, including sampling bias, data snooping and unrealistic splits.	A checklist, not a system. We follow it (validation-only tuning, temporal split, seeds).
07	Pendlebury, Pierazzi, Jordaney, Kinder, Cavallaro. <i>TESSERACT: Eliminating Experimental Bias in Malware Classification across Space and Time</i> . USENIX Security 2019.	Showed that random splits inflate results; argued that tests must be later in time than training.	About malware. We apply the same idea to network flows (temporal split).
08	Apruzzese, Laskov, Schneider. <i>SoK: Pragmatic Assessment of Machine Learning for Network Intrusion Detection</i> . IEEE EuroS&P 2023.	Re-evaluated ML detectors across many settings, attacks and hardware, with a practitioner study.	Static detectors; no continual learning, no graphs.

#	Paper	What it did	What it leaves open for us
09	Sarhan, Layeghy, Portmann. <i>Towards a Standard Feature Set for NIDS Datasets</i> . Mobile Networks and Applications 2022.	Proposed a common NetFlow feature set so datasets can be compared.	Feature standardisation; detection over time not studied.
10	Dragoi et al. <i>AnoShift: A Distribution Shift Benchmark for Unsupervised Anomaly Detection</i> . NeurIPS 2022 Datasets & Benchmarks.	Ten years of traffic (Kyoto-2006+) split into near and far future; measured how detectors decay.	Unsupervised, no attack categories learned in sequence, no graph.

5.2 Graph neural networks for intrusion detection

#	Paper	What it did	What it leaves open for us
11	Hamilton, Ying, Leskovec. <i>Inductive Representation Learning on Large Graphs (GraphSAGE)</i> . NeurIPS 2017.	The sample-and-aggregate GNN that works on graphs it has never seen.	General method, not security. It is the base of our model.
12	Lo, Layeghy, Sarhan, Gallagher, Portmann. <i>E-GraphSAGE: A GNN-based Intrusion Detection System for IoT</i> . IEEE/IFIP NOMS 2022.	Adapted GraphSAGE to classify edges (flows) with edge features; strong results on NetFlow datasets.	Trained once; no new attacks over time, no forgetting, no drift, no analyst layer. Our model's architecture.
13	Pujol-Perich, Suárez-Varela, Cabellos-Aparicio, Barlet-Ros. <i>Unveiling the Potential of GNNs for Robust Intrusion Detection</i> . 2021/22.	GNN on CIC-IDS2017 that stays accurate when attackers alter packet sizes and timings.	Static training; robustness to feature tampering, not to new attack types.
14	Caville, Lo, Layeghy, Portmann. <i>Anomal-E: A Self-Supervised NIDS based on GNNs</i> . Knowledge-Based Systems 2022.	Learns edge embeddings without labels, then detects anomalies.	No supervised continual learning; no incidents or explanations.
15	Bilot, El Madhoun, Al Agha, Zouaoui. <i>Graph Neural Networks for Intrusion Detection: A Survey</i> . IEEE Access 2023. (hal.science/hal-04099607 , bot check)	Survey of GNN-based IDS work.	Survey; identifies open problems rather than solving them.
16	Li, Zhang, Yuan, Zhou. <i>Network Intrusion Detection with Edge-Directed Graph Multi-Head Attention Networks</i> . arXiv 2023.	Attention-based GNN that uses edge direction.	Static benchmark accuracy.
17	Wang, Zheng, Gui, Hua, Hassan. <i>Are We There Yet? Unraveling the State-of-the-Art Graph Network Intrusion Detection Systems</i> . arXiv 2025.	Systematically re-evaluates published graph-based NIDS.	An evaluation study of existing graph detectors; continual learning and drift are outside its scope.
18	El Mahdaouy, Ait Yahia, Oualil, Berrada. <i>Deep Learning for Contextualized NetFlow-Based Network Intrusion Detection: Methods, Data, Evaluation and Deployment</i> . arXiv 2026.	Recent survey of detectors that use context beyond a single flow.	Survey; continual adaptation and analyst workload remain open.

5.3 Continual learning (general and on graphs)

#	Paper	What it did	What it leaves open for us
19	Kirkpatrick et al. <i>Overcoming Catastrophic Forgetting in Neural Networks</i> . PNAS 2017.	Introduced EWC: penalise changes to weights that were important for earlier tasks.	Not security; and EWC alone is known to struggle when new classes are added. We confirm that (0.300).
20	Rolnick, Ahuja, Schwarz, Lillicrap, Wayne. <i>Experience Replay for Continual Learning</i> . NeurIPS 2019.	Showed that replaying a small buffer of old data is a strong defence against forgetting.	Not graphs, not security. We replay whole traffic windows.
21	van de Ven, Tolias. <i>Three Scenarios for Continual Learning</i> . arXiv 2019.	Defined task-, domain- and class-incremental learning and showed class-incremental is hardest.	A framework. Our multiclass setting is class-incremental; our binary setting is domain-incremental.
22	Wang, Zhang, Su, Zhu. <i>A Comprehensive Survey of Continual Learning: Theory, Method and Application</i> . IEEE TPAMI 2024.	Broad survey of continual-learning methods and applications.	Survey; security applications only touched on.
23	Zhou, Cao. <i>Overcoming Catastrophic Forgetting in GNNs with Experience Replay (ER-GNN)</i> . AAAI 2021.	Replay for graph neural networks: choose which nodes to keep.	Node classification on citation-style graphs, not network flows.
24	Liu, Yang, Wang. <i>Overcoming Catastrophic Forgetting in Graph Neural Networks (TWP)</i> . AAAI 2021.	Protects weights that matter for the graph's topology, an EWC-like idea for GNNs.	Same: generic graphs, not intrusion detection.

5.4 Continual learning for intrusion detection

#	Paper	What it did	What it leaves open for us
25	Amalapuram, Tadwai, Vinta, Channappayya, Tamma. <i>Continual Learning for Anomaly based Network Intrusion Detection</i> . COMSNETS 2022. (paywalled; newslab.iith.ac.in)	Applied EWC and GEM to CICIDS and KDD; studied class imbalance and task order.	Per-flow models; no graph structure, no drift trigger, no analyst layer.
26	Amalapuram, Channappayya, Tamma. <i>Augmented Memory Replay-based Continual Learning Approaches for Network Intrusion Detection</i> . NeurIPS 2023.	Class-balanced replay and a cheaper way to pick replay samples, on many NIDS datasets including CIC-IDS2017/2018.	Focuses on the replay mechanism; graph context and analyst workload are complementary questions.
27	Zhang et al. <i>Continual Learning with Strategic Selection and Forgetting for Network Intrusion Detection</i> . IEEE INFOCOM 2025.	Chooses which new samples to keep and deliberately forgets outdated ones.	Focuses on which samples to keep; we study graph context, retraining timing and analyst output.
28	Banerjee, Bergqvist, Toor, Rohner, Johnsson. <i>Quantifying Catastrophic Forgetting in IoT Intrusion Detection Systems</i> . arXiv 2026.	Measured forgetting across continual-learning methods; found replay best overall.	IoT setting. Its replay finding agrees with ours in a different domain.

5.5 Concept drift: noticing change and adapting

#	Paper	What it did	What it leaves open for us
29	Bifet, Gavalda. <i>Learning from Time-Changing Data with Adaptive Windowing (ADWIN)</i> . SIAM SDM 2007.	The adaptive-window change detector with statistical guarantees.	Generic method. It is our drift trigger.
30	Andresini, Pendlebury, Pierazzi, Loglisci, Appice, Cavallaro. <i>INSOMNIA: Towards Concept-Drift Robustness in Network Intrusion Detection</i> . ACM AISEC 2021.	Semi-supervised NIDS that updates continuously, with active learning and label estimation.	A drift-robust per-flow detector; we add a direct comparison of a drift trigger against a fixed schedule for a graph model.
31	Barbero, Pendlebury, Pierazzi, Cavallaro. <i>Transcending Transcend: Revisiting Malware Classification in the Presence of Concept Drift</i> . IEEE S&P 2022.	Uses conformal evaluation to reject predictions that drift has made untrustworthy.	Malware, not network graphs.
32	Yang et al. <i>CADE: Detecting and Explaining Concept Drift Samples for Security Applications</i> . USENIX Security 2021.	Detects drifting samples with contrastive learning and explains them.	Detection of drift, not continual retraining of a graph detector.
33	Chen, Ding, Wagner. <i>Continuous Learning for Android Malware Detection</i> . USENIX Security 2023.	Active learning plus contrastive learning to keep a malware classifier current over seven years.	Malware; analyst labelling budget is studied, which inspired our label-budget experiment.

5.6 Trust: calibration, uncertainty, novelty, analysts

#	Paper	What it did	What it leaves open for us
34	Guo, Pleiss, Sun, Weinberger. <i>On Calibration of Modern Neural Networks</i> . ICML 2017.	Showed modern networks are overconfident; introduced ECE measurements.	Generic; we measure our model's calibration (ECE 2.2 % on attacks).
35	Angelopoulos, Bates. <i>A Gentle Introduction to Conformal Prediction and Distribution-Free Uncertainty Quantification</i> . 2021.	Tutorial on prediction sets with coverage guarantees.	Generic method; we apply its class-conditional form for abstention.
36	Liu, Wang, Owens, Li. <i>Energy-based Out-of-distribution Detection</i> . NeurIPS 2020.	The energy score for spotting inputs unlike the training data.	Images; we use it as one of three novelty scores for traffic.
37	Alahmadi, Axon, Martinovic. <i>99% False Positives: A Qualitative Study of SOC Analysts' Perspectives on Security Alarms</i> . USENIX Security 2022.	Interviewed SOC practitioners; alarms must be reliable, explainable, analytical, contextual and transferable.	A user study, not a system. It motivates our incidents, explanations and abstention.
38	Barrett, Li, Dorai, Rajaganapathy. <i>FIRCE: A Framework for Intrusion Response and Conformal Evaluation</i> . arXiv 2026.	Adds conformal uncertainty and drift detection to supervised IDS classifiers.	Adds uncertainty to existing IDS classifiers; not combined with a continual graph model.

5.7 Tools the project is built on

#	Paper	What it did	Role here
39	Fey, Lenssen. <i>Fast Graph Representation Learning with PyTorch Geometric</i> . ICLR Workshop 2019.	The PyTorch library for GNNs.	Graph data structures and message passing.
40	Chen, Guestrin. <i>XGBoost: A Scalable Tree Boosting System</i> . KDD 2016.	Gradient-boosted decision trees.	The tree-ensemble baseline (the Random-Forest family).

6 · The gap: what was missing, and what this project does about it

Each line of work above solves one piece. In the papers reviewed here we did not find one system that does all of the pieces together *and* evaluates them with a time-ordered split, several seeds and statistics. That combination, and the honesty of the evaluation, is what this project contributes. It is a combination of known techniques, not a new algorithm, and should be presented that way.

What is missing	Who it affects	What this project does	Evidence
GNN detectors are trained once. No new attacks over time, no forgetting measured.	12, 13, 14, 16	Graph model learns 7 attack types in sequence with replay and EWC; retention, BWT and per-category recall reported after every task.	2 datasets; 3–5 seeds
Continual NIDS uses per-flow models. Host relationships are ignored.	25, 26, 27, 28	The same continual-learning recipe applied to a graph model <i>and</i> to an identical per-flow model, so the graph's effect is isolated.	Paired statistics, both splits
Unseen attacks are rarely tested for continual detectors.	25–28	Leave-one-attack-out: graph vs per-flow vs XGBoost on attacks never trained on.	2 seeds (2018), 1 seed (2017)
When to retrain is rarely compared with simple alternatives.	30, 31, 33	ADWIN vs fixed schedule vs perfect-knowledge oracle vs never, on clean and mixed attack streams.	3 seeds each
Optimistic splits and single runs.	6, 7, 17	Time-ordered split per attack, 5 seeds, bootstrap confidence intervals, Wilcoxon tests; error-corrected datasets.	results/stats
Detectors stop at flow-level accuracy. Analyst workload is ignored.	37	Incidents, explanations, conformal abstention, calibration, novelty scores, dry-run response with approve/reject.	Single runs, labelled
Results are hard to reproduce.	6, 17	Every number written by a script; one command regenerates them; 92 tests; console reads the same files.	README, CI

What we do *not* claim

- That the graph model is always better: on attacks it has seen, the gain is small (+0.04) and the per-flow model has a lower false-positive rate on one split.
- That unseen-attack detection is reliable for every attack: BruteForce and Botnet are unstable across three seeds (DoS, however, is detected at 98.6 % on all three).
- That ADWIN saves retraining: on CIC-IDS2017 it retrains about twice as often as a fixed schedule.
- That lab results transfer to a real network: both datasets are laboratory captures.

7 · The machine-learning model: why a graph network, and why not Random Forest

7.1 The model we use, in plain words

The deployed model is called `gnn_ewc_replay`: an **E-GraphSAGE graph neural network** trained with **EWC** and **replay**. Here is what happens to one flow, step by step:

1. **Build the graph.** The 5,000 flows of a window become a graph. Each computer is a node; each flow is an edge carrying its 83 numbers (scaled so that big and small numbers are comparable). Each node also gets simple structural numbers: how many flows it sends and receives.
2. **Look around (two hops).** For each computer the model gathers a summary of the flows going out of it and the flows coming into it, separately (direction matters: a scanner sends, a victim receives). It does this twice, so a computer's summary also reflects its neighbours' neighbours.
3. **Decide per flow.** For each flow, the summaries of its two endpoints are joined with the flow's own numbers (the "edge skip", so the flow itself is never drowned out), and a small classifier outputs a probability for benign and for each attack type.

Hidden size / layers	64 / 2 (two hops of neighbourhood)
Parameters	69,128 (the per-flow FFNN ablation has 55,432)
Training per task	10 epochs, Adam, learning rate 0.001, class weights for rare attacks (chosen on the validation split, never on test)
EWC	$\lambda = 10$, $\gamma = 0.9$ (chosen by a validation sweep over λ and γ)
Replay	up to 10 stored windows per attack category, 2 replayed at every training step

7.2 The models it is compared against

The project keeps every baseline and ablation, because they are the evidence for the design choices:

Model	What it is	The question it answers
XGBoost static	Boosted decision trees, trained once on the first task, never updated	What does a classic tree model do when it never learns new attacks?
GNN naive retrain	Our graph model retrained on each new task with no protection	How bad is forgetting?
FFNN + EWC + replay	A plain neural network judging each flow alone, with the <i>same</i> continual-learning recipe	Does the graph help? (only the graph differs)
GNN + replay only	Our model without EWC	Does EWC add anything?
GNN + EWC only	Our model without replay	Does replay matter?

Model	What it is	The question it answers
GNN / FFNN joint	Retrained on all data seen so far, every time	The upper bound a continual model aims for

7.3 Why not Random Forest (or any per-flow tree model)?

You mentioned an early Random Forest version that was dropped. It is not in this repository's history, which starts on 17 September 2026, so this guide explains the decision using the tree-ensemble baseline the repository does contain: XGBoost. Random Forest and XGBoost are close relatives: both are many decision trees voting, and both judge each flow on its own.

Random Forest is a good classifier and a sensible first attempt. It was replaced because the product needs four things a per-flow tree model cannot provide:

Need	Random Forest / XGBoost	Graph neural network with continual learning
See relationships between computers (scans fan out, floods fan in)	Judges one row at a time; "this host talked to 996 ports" is invisible unless you hand-craft extra features.	Reads the neighbourhood directly from the graph.
Catch attacks never trained on	Our XGBoost baseline caught 0 % of five of the six never-seen 2018 attacks (90.1 % on DoS only); the per-flow neural network caught at most 0.07 % on all of them.	More than 80 % in 6 of 12 held-out tests.
Learn new attacks over time without starting from scratch	Trees are rebuilt from scratch; to keep old knowledge you must keep and retrain on <i>all</i> old data. A tree model trained once (our XGBoost static) ends at 0.231 macro-F1 because it never learns the later attacks.	Updates on the new task, rehearsing only a small replay buffer, and keeps 99.8–100 % retention.
Say "I'm not sure" and "this is new"	Tree vote fractions are poorly calibrated probabilities; no learned embedding for distance-based novelty.	Softmax probabilities support conformal abstention, and the hidden layer gives embeddings for novelty and drift scores.

Where Random Forest would be the right choice: a fixed set of known attacks, no need to learn over time, and a need for very fast CPU-only training with feature importances out of the box. That is not this product's situation.

7.4 How the model choice benefits the product

- **Incidents come naturally.** Because the model already thinks in hosts and connections, grouping flagged flows into attacker/victim incidents uses the same graph.
- **Better explanations.** The explanation can say "80 % of the evidence came from neighbouring flows", which tells an analyst the graph context mattered (see the incident detail screenshot).
- **Fewer missed new attacks.** Structural patterns transfer to attacks the model never saw, at least for some of them.

- **Cheap to keep current.** Each update trains on the new traffic plus a small buffer, in seconds per task on a laptop GPU, instead of retraining on everything.

7.5 What the numbers say about the model (CIC-IDS2017, final task)

Model	Interleaved split (3 seeds)	Temporal split (5 seeds)	Remembers first attack
GNN + EWC + replay (ours)	0.964 ± 0.020	0.915 ± 0.031	100 % / 99.8 %
FFNN + EWC + replay (no graph)	0.928 ± 0.020	0.870 ± 0.009	100 % / 99.7 %
GNN + replay only	0.948 ± 0.016	0.848 ± 0.089	100 % / 99.4 %
GNN + EWC only	0.300	0.377	0 %
GNN naive retrain	0.289	0.263	0 %
XGBoost static (tree model)	0.231	0.236	100 % / 99.0 % (it never learns anything new)
GNN joint retrain (upper bound)	0.951	0.935	100 % / 99.7 %

8 · The data: why CIC-IDS2017 and CSE-CIC-IDS2018

8.1 The two datasets

	CIC-IDS2017	CSE-CIC-IDS2018
Made by	Canadian Institute for Cybersecurity (Sharafaldin et al., 2018)	Communications Security Establishment and CIC
Network	A lab office network, five working days	A larger network hosted on AWS, about ten days
Release used	Error-corrected by Engelen et al. (2021)	Error-corrected by Liu et al. (2022)
Attack tasks, in capture order	BruteForce → DoS → WebAttack → Infiltration → Botnet → PortScan → DDoS (7)	BruteForce → DoS → DDoS → WebAttack → Infiltration → Botnet (6)
Flows used	All	A 15 % sample, taken without looking at labels (the full set is too large for a 24 GB laptop)
Features	The same CICFlowMeter flow features (83 used), so one model design fits both	

8.2 Why these two, and not more recent ones?

Newer intrusion datasets exist (for example CIC-DDoS2019, BoT-IoT, TON_IoT and CIC-IoT2023). These two were chosen because the project needs a very specific combination, and they are the best-documented datasets that have it:

1. **Many different attack types arriving one after another, with timestamps.** Continual learning needs a natural *sequence* of new attacks. Both datasets recorded different attacks on different days, so the capture schedule gives a realistic order of tasks. Datasets built around one attack family (such as a DDoS-only set) cannot provide that.
2. **Source and destination addresses kept.** A graph needs to know who talked to whom. Some re-released datasets anonymise or drop addresses, which makes graph building impossible.
3. **Corrected versions exist.** Researchers found and fixed thousands of labelling and flow errors in exactly these two datasets (papers 02 and 03). A detector evaluated on wrong labels gives wrong numbers (paper 04). Very few newer datasets have had this level of independent checking.
4. **Comparability.** They are the most-used enterprise intrusion benchmarks, including by the graph and continual-learning papers this project builds on (12, 13, 26), so the results can be placed next to published work.
5. **Enterprise rather than IoT traffic.** The target user is an organisation's security team. Many newer datasets focus on IoT devices, a different traffic profile.

The honest downside: 2017 and 2018 traffic is old, and it is laboratory traffic with attacks in clean blocks. The project tests one consequence of that directly (the mixed-attack stream, section 12) and lists validation on newer and real traffic as future work.

8.3 Why two datasets and not one?

Because one dataset can fool you. A result that holds on one network might be an accident of that network's particular hosts, timings or labelling. The second dataset is a **different network** (bigger, cloud-hosted, a different attack order) measured with the **same features**, so the same model can be tested on both. That is replication, and it changed conclusions:

- EWC alone looked fine in 2017 binary mode but **failed on 2018**, which is part of why the project concludes that replay, not EWC, is what prevents forgetting.
- The drift detector over-triggered on 2017 but **beat a fixed schedule on 2018** (one run): its value depends on the network.
- The graph's advantage on attacks it has seen is clear on 2017 but unstable on 2018, where the same configuration scored 0.855 in one training and 0.945 in another.
- The strongest graph result, detection of never-seen attacks, is **strongest on 2018**.

8.4 How the data become tasks and test sets

Flows are sorted by time and cut into windows of 5,000. Each window belongs to the attack task during which it was captured. Each task's windows are split into training, validation and test. Two ways of splitting are reported:

Interleaved (by position in groups of 10)



Temporal (first 70 % train, gap, 10 % validation, gap, last 20 % test)



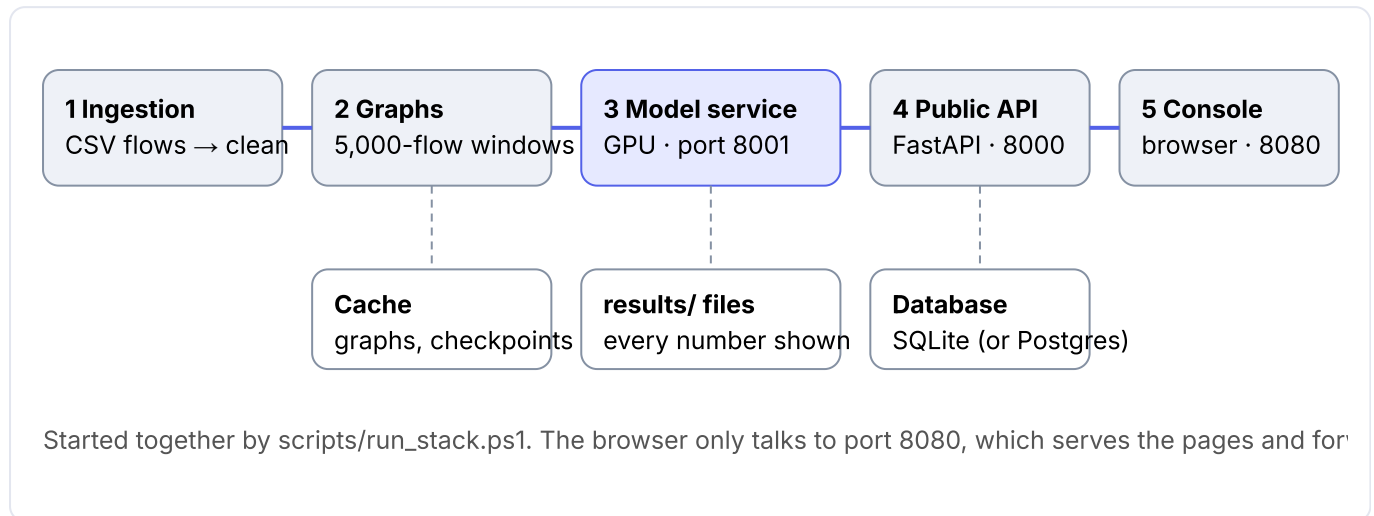
■ train ■ validation ■ test ■ gap (dropped)

Interleaved tests sit minutes away from training windows of the same attack session, which is optimistic. Temporal tests are always later in time, which is realistic.

A plain time split leaves some attacks with nothing to test on (WebAttack and Botnet would get 0 test attack flows, DoS only 11), because those attacks sit at one end of their task. So the project uses a **per-attack temporal split**: the windows carrying the new attack and the other windows are each split 70/10/20 in time. Every attack is then tested on its own latest traffic. This cost our model 0.049 macro-F1 (0.964 → 0.915), which is the honest price of a realistic test.

9 · How the product works, end to end

9.1 The five layers



9.2 Following one window through the system

1. **Ingestion** reads the corrected CSV files, renames columns to a standard form, fixes infinite or missing values, relabels "attempted" attacks (no malicious payload) as benign as the dataset authors advise, and assigns each flow to its attack task by time.
2. **Preprocessing** applies a log transform and scaling learned on training data only, clips extreme values, and cuts the stream into windows of 5,000 flows.
3. **Graph building** turns each window into a graph (hosts → nodes, flows → edges) and caches it on disk, so later steps never repeat this work.
4. **Training** (offline, experiments/run_continual.py) learns the tasks in order and saves a checkpoint of every model after every task.
5. **Serving**: the model service loads the final checkpoints. When the console asks about window 363, it scores all 5,000 flows in about 5.5 ms on the GPU and caches the answer.
6. **Incidents**: flows predicted as attacks are linked when they share hosts, giving one group per attacker/victim cluster and category. Each group gets a severity (size × confidence), a key host, an explanation and a proposed action.
7. **Human decision**: the analyst approves or rejects. The decision is stored in the database with their name and a note. Nothing is executed.

9.3 The live loop (Live stream page)

The same steps run continuously during a replay. For each new window, each of four models predicts first. Then the true labels are revealed (the "delayed labels" an analyst would provide), the error goes to ADWIN, and if ADWIN confirms a rise in error the model adapts on the last 20 windows using its continual-learning strategy. Everything is written to the database and shown live.

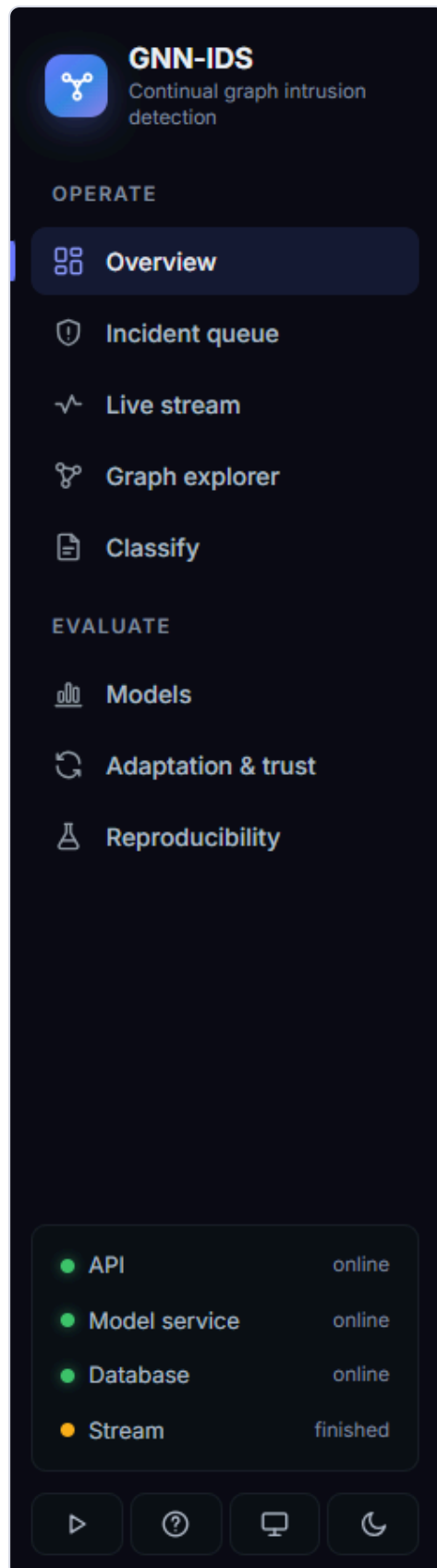
9.4 Why it is split into separate services

- The heavy model work (GPU, PyTorch) runs in its own process, so the public API stays responsive.
- If the model service is down, the pages that read saved results still work, and a banner names the pages that need the model service.
- The browser never talks to the model service directly. It only talks to port 8080, which is easier to secure.

10 · Every page and every feature of the console

The console is a website that runs in the browser at `http://localhost:8080`. It has eight pages in two groups: **Operate** (what a security team uses day to day) and **Evaluate** (the evidence that the model works). Every number on screen is read from a result file or from the live model; nothing is typed in by hand. Screenshots were taken from the running system with the model service up, in the dark theme at 1440×900.

10.1 The frame around every page



The sidebar: brand, the eight pages, service status, and four utility buttons.

Navigation

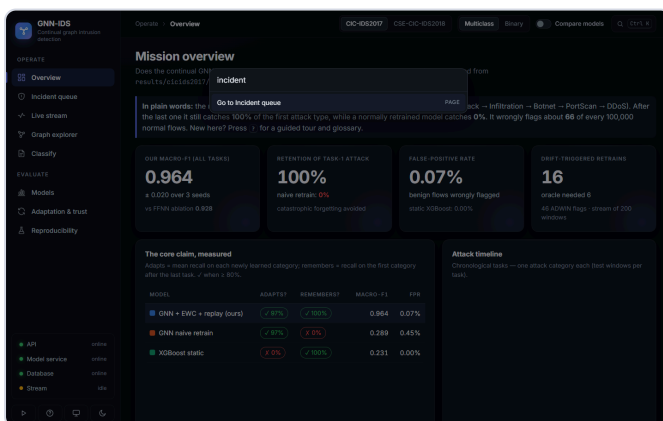
One item per page with an icon. The current page is highlighted with an accent bar. Hovering an item shows its keyboard number (1–8). A red **LIVE** badge appears on Live stream while a replay runs.

Service status	Four rows checked every 4 seconds: API (port 8000), Model service (port 8001, the GPU), Database , and Stream (idle, finished, or progress such as 120/200). Green dot = online, red = offline. Hovering Model service shows the device (cuda + GPU name) and the models loaded.
► Tour	A five-step guided walk: Overview → Incident queue → Live stream → Graph explorer → Models, one sentence each.
? Help	Opens a drawer with "where to start", keyboard shortcuts and a glossary. It opens by itself on a first visit.
Presentation mode	Larger text and thicker chart lines for a projector; fine print is hidden.
Theme	Switches dark and light. Charts redraw in the new colours.

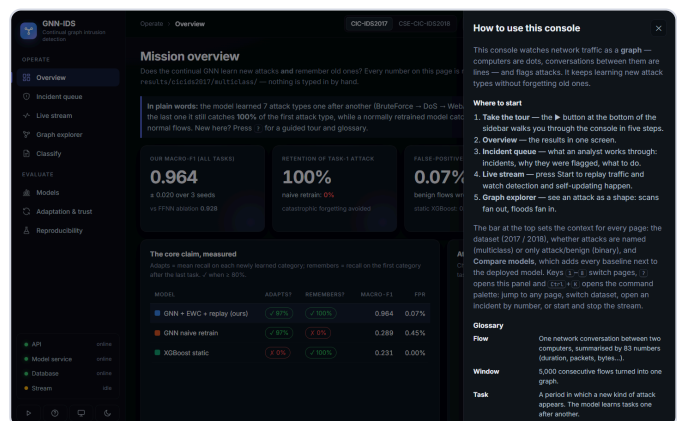


The top bar: where you are, and the three settings shared by every page.

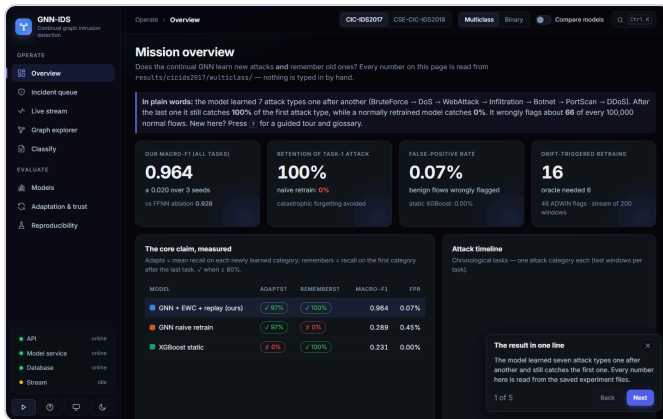
Breadcrumb	The group and page name, e.g. "Operate > Overview".
Dataset	CIC-IDS2017 or CSE-CIC-IDS2018. Every open page re-reads its data.
Labels	Multiclass names each attack type; Binary is only attack vs benign. Binary is disabled for CSE-CIC-IDS2018, because that experiment had only one seed and was withdrawn (hover explains why).
Compare models	Off: charts show our model and two reference points (naive retraining, XGBoost). On: every baseline and ablation appears.
Jump to... (Ctrl K)	The command palette: type to jump to a page, switch dataset or labels, start or stop the stream, switch theme, or type a number to open that incident.



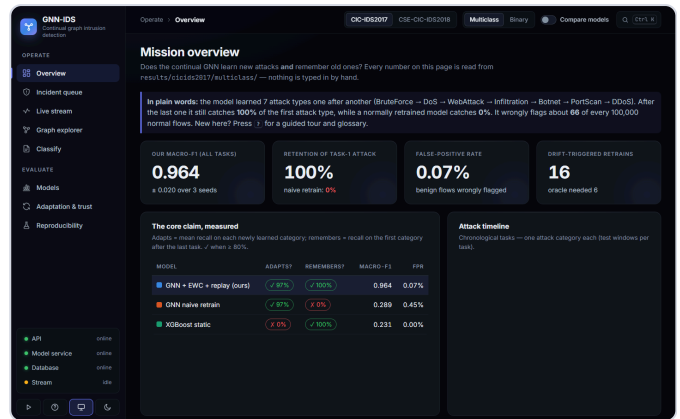
Command palette: typing "incident" offers the Incident queue.



Help drawer with the glossary.



The guided tour card (bottom right).

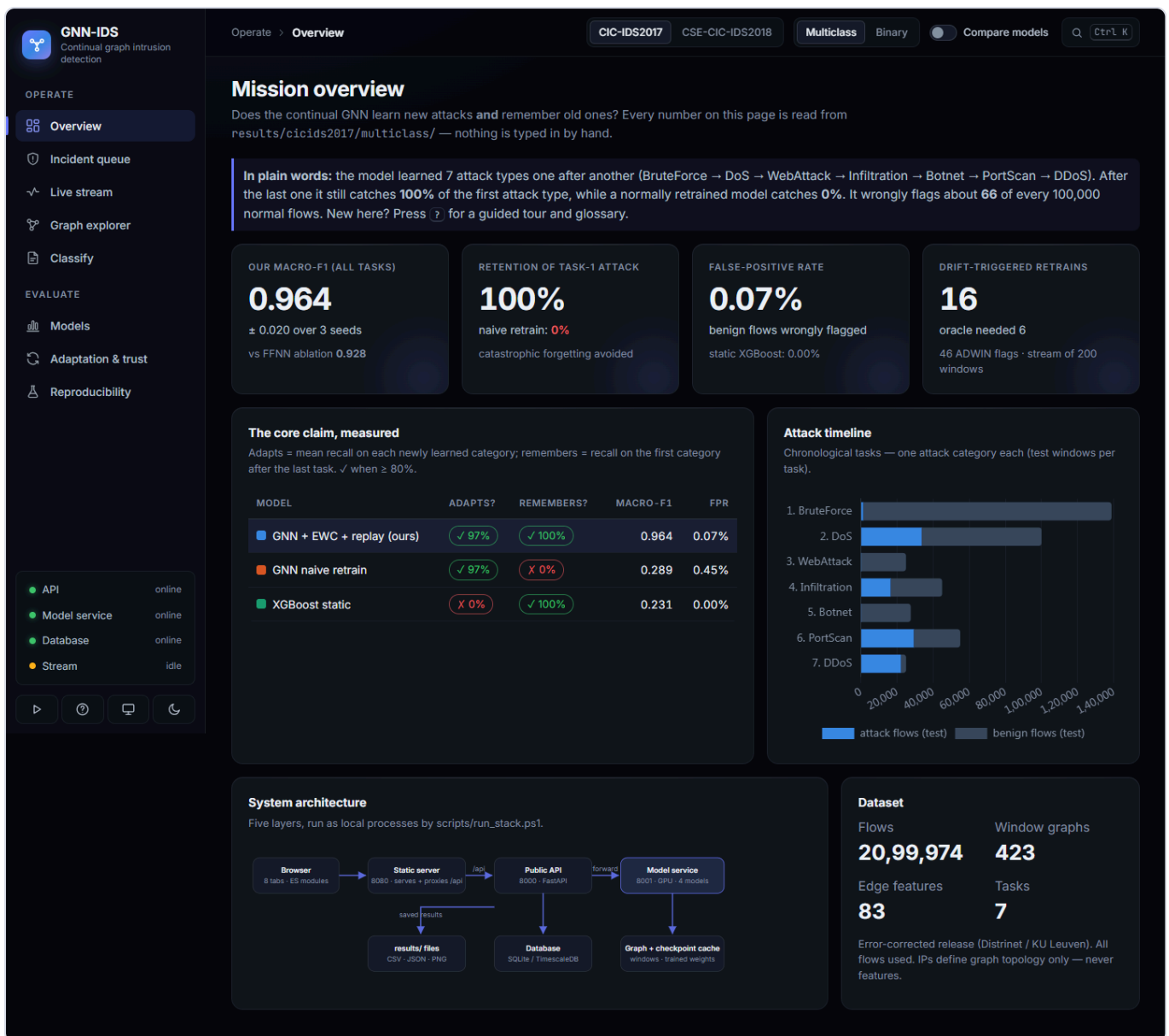


Presentation mode: bigger type for a projector.

Keyboard: 1–8 switch pages, ? opens help, Esc closes panels, Ctrl+K opens the palette. In the Incident queue: j / k move, Enter opens, a approves, r rejects, c copies the rule.

Other details: a yellow **banner** appears under the top bar when the model service is down, naming the pages that still work; ↓ **export buttons** on panels download the data as CSV (and charts as PNG); preferences (theme, dataset, labels, compare, chosen sub-page) are remembered by the browser; if the server requires an API key the console asks once and sends it in a header, never in the address.

10.2 Overview: "does it work?" in one screen



Overview, CIC-IDS2017 multiclass.

- **In plain words:** a sentence generated from the result files, e.g. "the model learned 7 attack types one after another... after the last one it still catches 100 % of the first attack type, while a normally retrained model catches 0 %. It wrongly flags about 66 of every 100,000 normal flows."
- **Four KPI tiles:** our macro-F1 (0.964 ± 0.020 over 3 seeds, versus 0.928 for the per-flow FFNN), retention of the first attack (100 %, versus 0 % for naive retraining), false-positive rate (0.07 %), drift-triggered retrains (16, where the oracle needed 6).
- **The core claim, measured:** per model, does it *adapt* (recall on each new category) and does it *remember* (recall on the first category at the end)? A tick appears at 80 % or more; the rule is written on the panel.
- **Attack timeline:** attack and benign test flows per task, in capture order.
- **System architecture** diagram and a **Dataset** panel (flows, window graphs, features, tasks).

Example use: an examiner asks "does it forget?". Open Overview, point at the Retention tile (100 %) and the red 0 % for naive retraining underneath it.

10.3 Incident queue: what an analyst works through



GNN-IDS

Continual graph intrusion detection

OPERATE

Overview

Incident queue

Live stream

Graph explorer

Classify

EVALUATE

Models

Adaptation & trust

Reproducibility

APIonline

Model serviceonline

Databaseonline

Streamidle

▶

🔄

💬

🌙

Operate

Incident queue

CIC-IDS2017

CSE-CIC-IDS2018

Multiclass

Binary

Compare models

🔍 Ctrl K

Incident queue

Thousands of flagged flows become a handful of incidents — one per attacker/victim cluster. Open one to see why it was flagged and the containment we would propose. You approve or reject; nothing is ever executed.

live models: cicids2017 · multiclass

Flows in window

5,000

network conversations analysed

Flagged flows

4,986

99.7% of the window

Incidents

1

≈ 4,986 alerts folded into each

Incidents that are real

100%

1 of 1 · checked against ground truth (demo only)

What to review

One window

Recent

Traffic window

#363 · PortScan · 4,986...

Detector

GNN + EWC + replay (...)

Minimum confidence

0%

Load incidents

Filter the queue

Category

All categories

Severity

Any severity

Export

↓ CSV

↓ CEF

Keyboard: **j** / **k** move, **Enter** open, **a** approve, **r** reject, **c** copy rule.

Queue 1

most severe first · severity = size × confidence

#1

PortScan · from 172.16.0.1

4,986 flows · 1 source(s) → 1 destination(s) · 100% confident · real attack

critical

Incident #1 · PortScan

critical

2017-07-07 17:52:40 → 17:52:47 · key host 172.16.0.1 (source, 4,986 flows)

Why was this flagged?

Flagged as PortScan with 100% confidence. The source probed 996 different destination ports (4986 flows) — a port-scan pattern. Strongest evidence from the flow itself: fwd packet length mean = 0, total length of fwd packet = 0, fwd segment size avg = 0. 80% of the attribution came from 6+ neighbouring flows, i.e. the graph context mattered.

What pushed the decision — red towards the verdict, blue against it

destination port

fwd packet length mean

total length of fwd packet

fwd segment size avg

subflow fwd bytes

fwd packet length max

Network context

FACT	VALUE
Source host talks to	1 hosts
Flows from source	4,986
Distinct ports it targeted	996
Destination hears from	1 hosts
Flows to destination	4,986
Evidence from neighbouring flows	80%

Example flow 172.16.0.1 → 192.168.10.50 · model says PortScan (100%) · ground truth PortScan

Proposed containment

DRY RUN

Block the attacking host — 172.16.0.1

172.16.0.1 originated 4986 flagged PortScan flows to 1 host(s); block it at the perimeter pending investigation.

Linux (iptables)

Windows Firewall

iptables -I INPUT -s 172.16.0.1 -j DROP

Copy rule

Open report

Decision

your name

note (optional)

Approve a

Reject r

Approving records the decision only. Connecting approved rules to a real firewall is a deployment choice deliberately left out of this project.

Decision log

every proposed action and the analyst's decision · dry run only, nothing was executed

All

Pending

Approved

Rejected

WHEN (LOCAL TIME)	WINDOW / INCIDENT	CATEGORY	ACTION	TARGET	DECISION	BY	NOTE
19/9/2026, 7:25:38 pm	#363 / 1	PortScan	Block the attacking host	172.16.0.1	approved	demo-analyst	

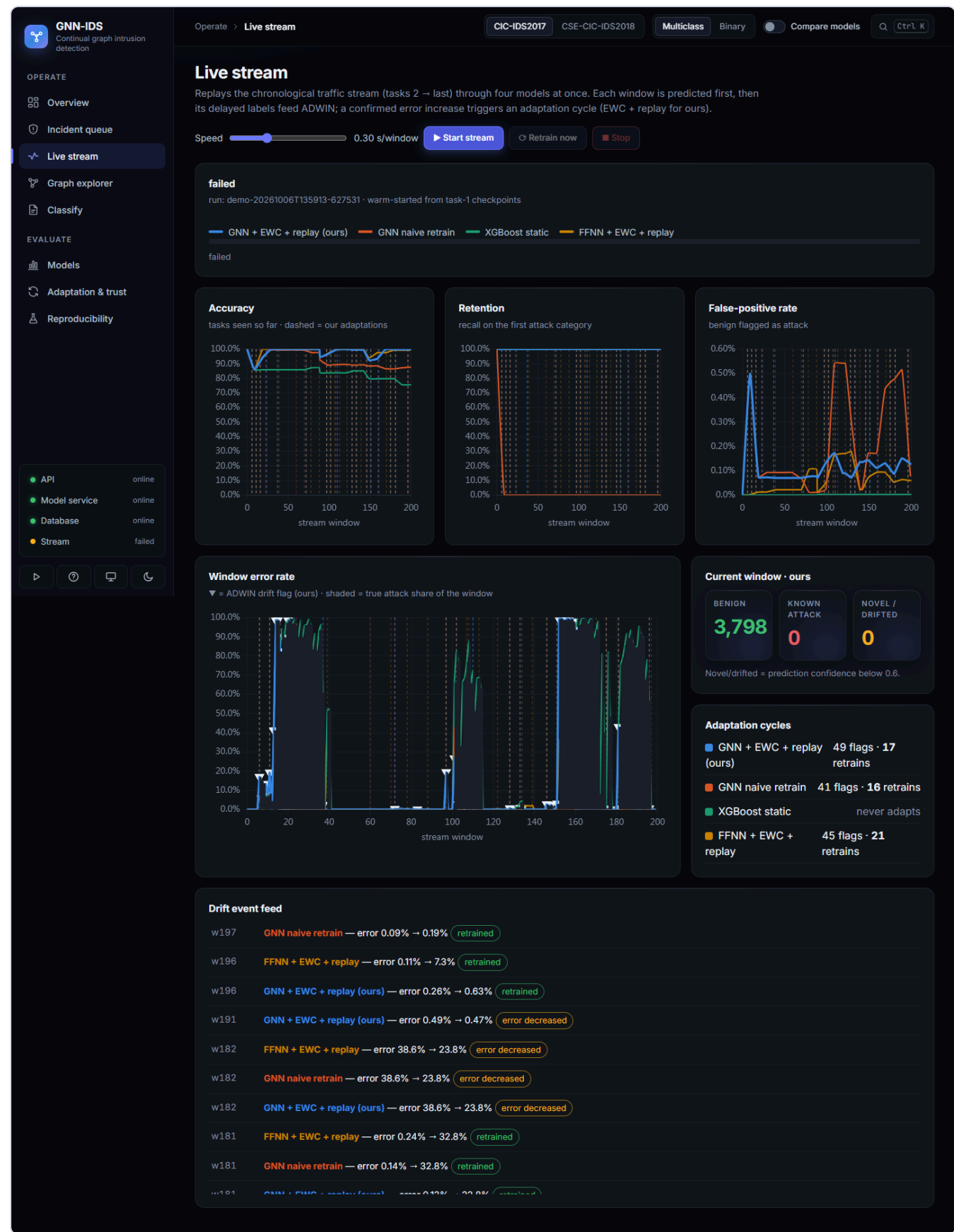
Window #363 (PortScan) loaded with our model; incident #1 open.

Why it exists: analysts cannot read thousands of flow alerts. This page groups them into incidents, explains each one, proposes a response and records the human decision.

What to review	One window (pick a traffic window) or Recent (scan the last 10/20/50 windows and rank incidents across them, like a shift's queue).
Detector	Which model scores the traffic: ours, the per-flow FFNN, or naive retraining.
Minimum confidence	Alerts below this confidence are dropped before grouping.
Load incidents	Scores the window and builds the queue.
KPI row	Flows in the window (5,000), flagged flows (4,986 = 99.7 %), incidents (1: "about 4,986 alerts folded into each"), and the share of incidents that are real (checked against ground truth, demo only).
Queue	Incidents ranked by severity (size × confidence), with category colour, key host, flow count, confidence and a severity bar. Filters by category and severity; export as CSV or as CEF lines for a SIEM.
Why was this flagged?	A plain-English explanation built by rules (no language model): "The source probed 996 different destination ports (4,986 flows), a port-scan pattern... 80 % of the attribution came from 6+ neighbouring flows, i.e. the graph context mattered."
What pushed the decision	A bar chart of feature attribution (gradient × input): red bars pushed towards the verdict, blue against.
Network context	Facts: hosts the source talks to, flows from source, distinct ports targeted, how many hosts the destination hears from, share of evidence from neighbouring flows.
Proposed containment (DRY RUN)	Block the attacking host, rate-limit to a victim, or isolate a host, chosen from the incident's shape, with the exact rule for Linux (iptables) or Windows Firewall , a Copy rule button and a printable one-page report .
Decision	Name, optional note, Approve or Reject . It is stored with a timestamp. Nothing is ever executed.
Decision log	Every proposal and decision, filterable by All / Pending / Approved / Rejected.

Example: window #363 contains 5,000 flows; 4,986 are flagged; they collapse into **one** incident: host 172.16.0.1 scanning 996 ports of one server. The proposed rule is `iptables -I INPUT -s 172.16.0.1 -j DROP`. The analyst types their name and presses Approve; the log records it as approved (dry run).

10.4 Live stream: watch it detect and adapt

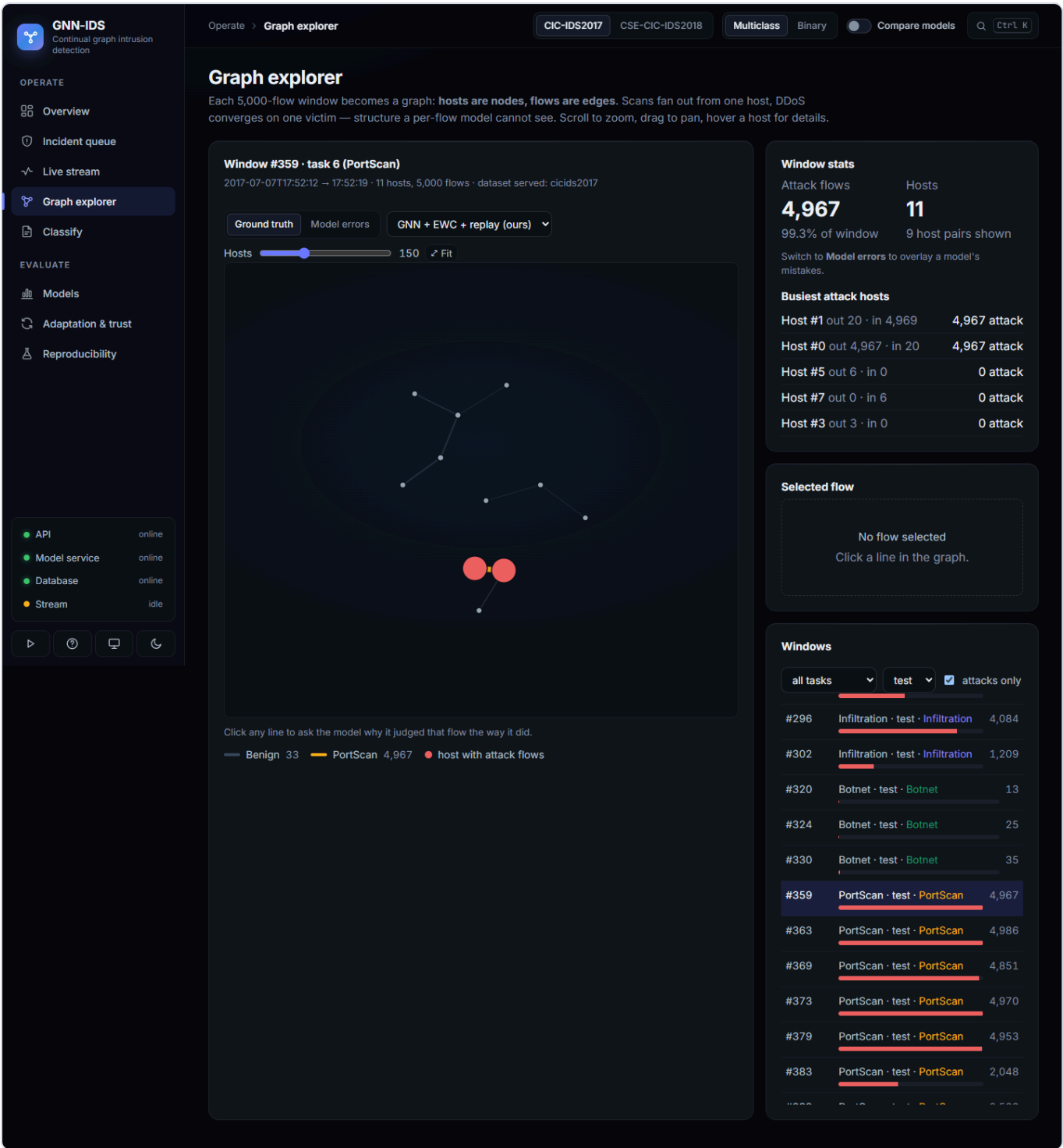


A recorded 200-window replay of tasks 2–7 through four models. (The status line at the top read "failed" when this was captured because of an unrelated GPU fault during the screenshot session; the charts are the completed run.)

Speed / Start / Retrain now / Stop	Delay between windows; start the replay; force an adaptation (operator override); stop.
Accuracy, Retention, False-positive rate	Live charts for all four models; dashed vertical lines are our model's adaptations.
Window error rate	Error per window; ▼ marks an ADWIN drift flag; the shaded area is the true attack share of the window.
Current window	Our model's verdicts on the latest window: benign, known attack, and novel/drifted (confidence below 0.6).
Adaptation cycles	Drift flags and retrains per model (e.g. ours 49 flags, 17 retrains in this run; XGBoost never adapts).
Drift event feed	One line per event: window, model, error before → after, and whether it retrained or the error decreased.

Example: around window 181 a new attack arrives and the error jumps; ADWIN flags it and the models retrain. The feed shows lines such as "w181 FFNN + EWC + replay — error 0.24 % → 32.8 % · retrained", and the error falls back over the next windows.

10.5 Graph explorer: an attack has a shape



Window #359 (PortScan): 5,000 flows between 11 hosts; two hosts carry 4,967 attack flows between them.

Ground truth / Model errors	Colour lines by their true category, or by whether the chosen model got them wrong (false alarms and missed attacks become visible).
Model selector	Which model's mistakes to overlay.
Hosts slider / Fit	Maximum hosts drawn (keeps busy windows readable) and re-centre the view.
Window stats	Attack flows, hosts, and the busiest attack hosts with flows out / in.

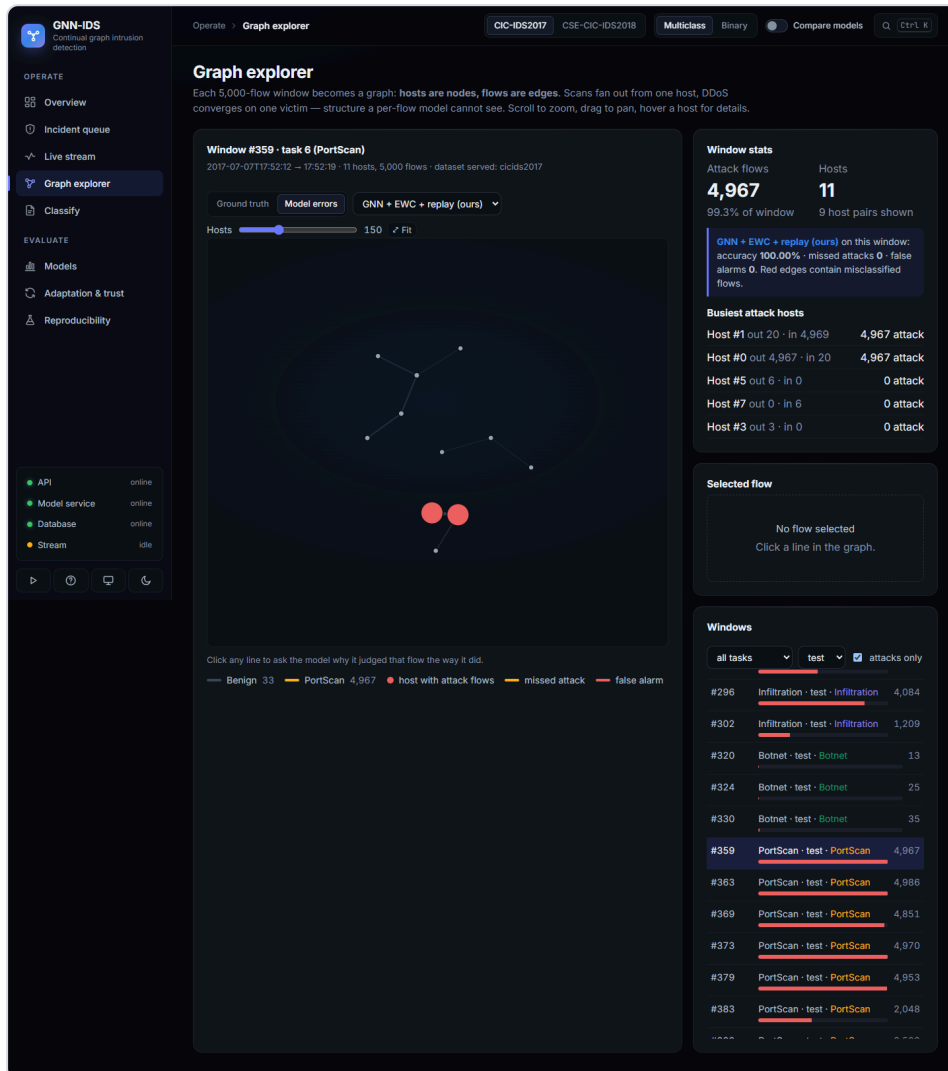
Selected flow

Click any line to ask the model why it judged that flow the way it did.

Windows list

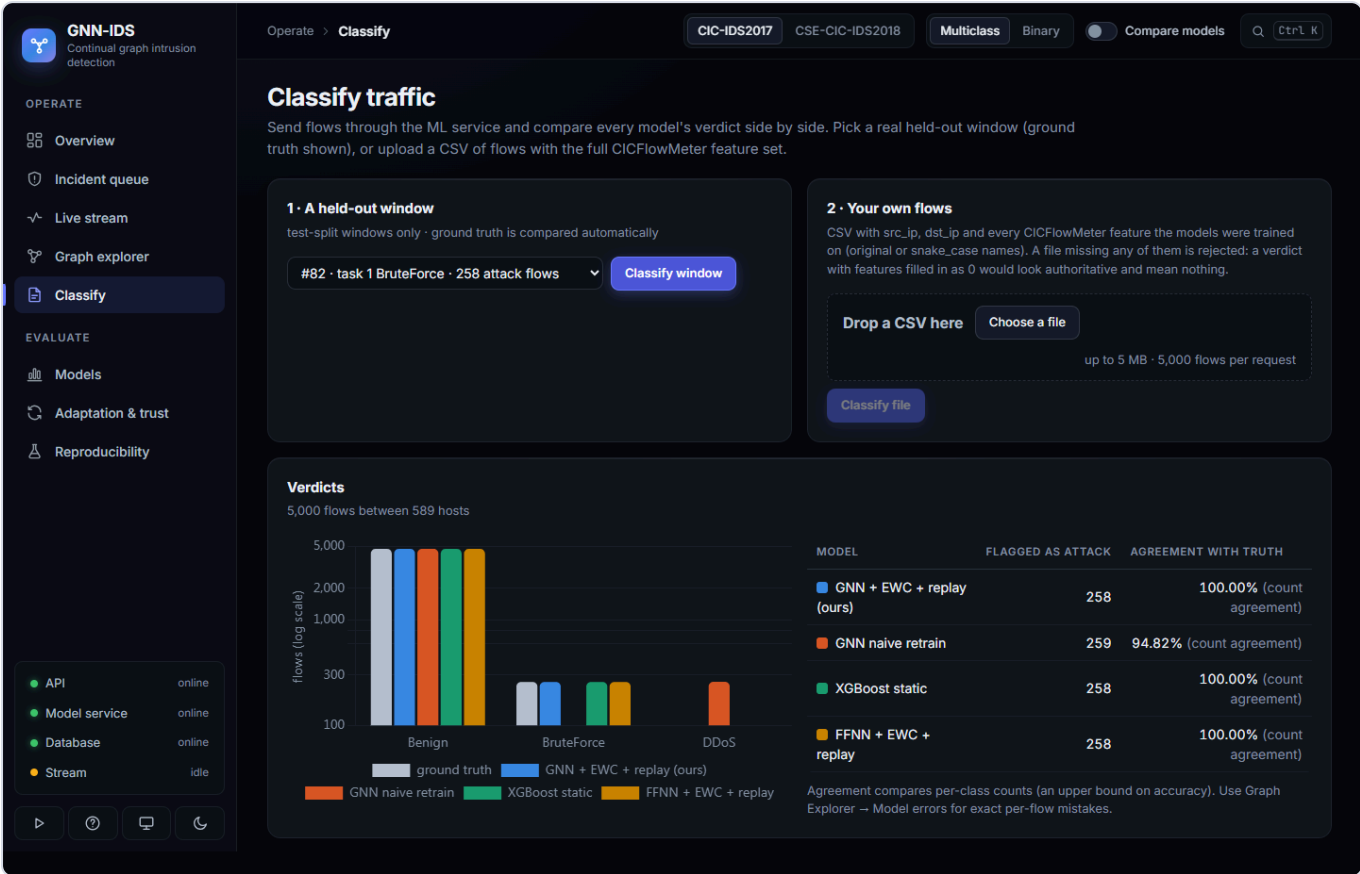
Filter by task, split (train/validation/test) and "attacks only"; each row shows the window number, task, category and attack-flow count with a bar.

Interaction: scroll to zoom, drag to pan, hover a host for its degree. Hosts with attack flows are drawn in red.



The same window coloured by model errors.

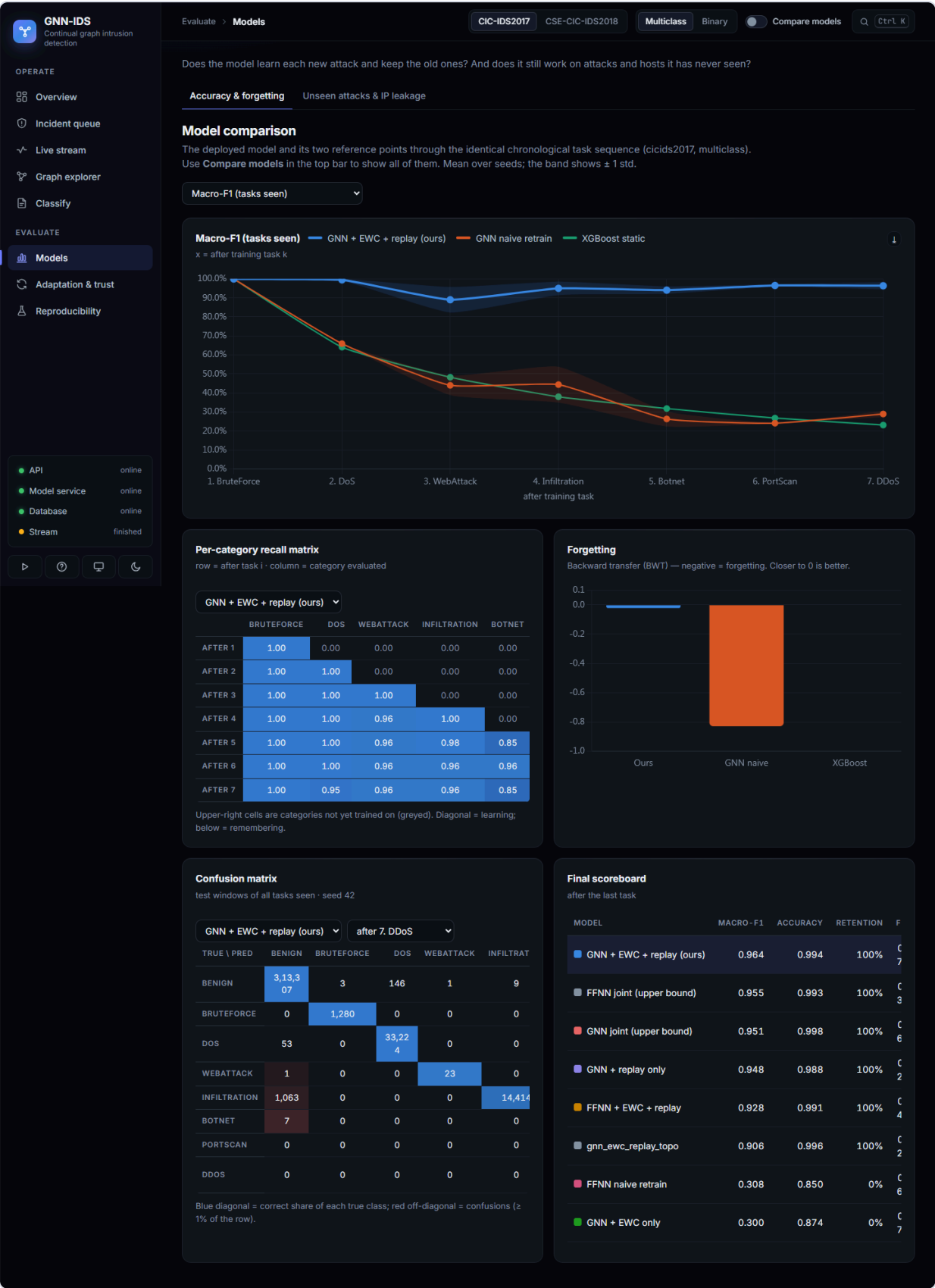
10.6 Classify: run the models on demand



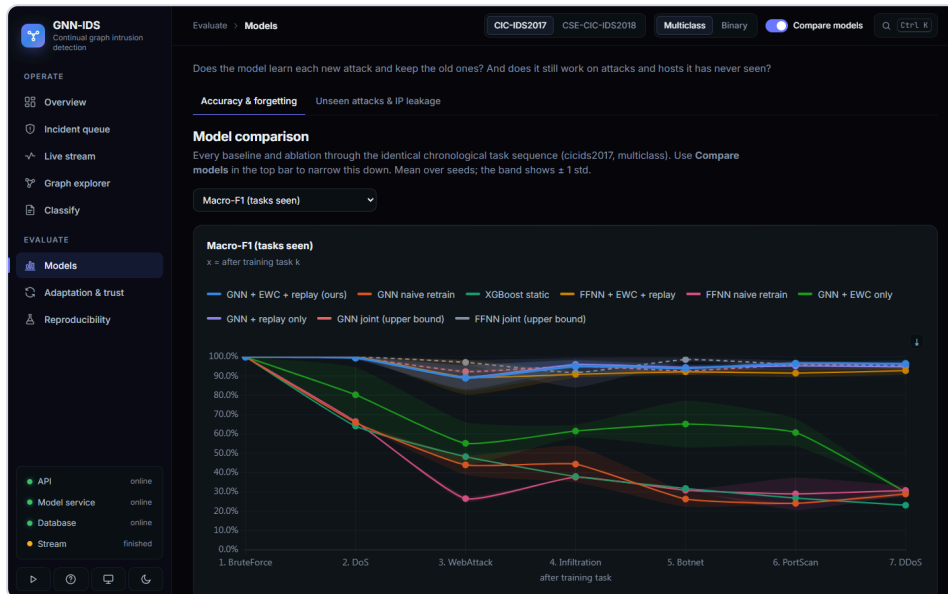
Held-out window #82 (BruteForce): every model's verdict compared with the truth.

- **A held-out window:** choose a test window and press Classify window. Each loaded model labels all 5,000 flows; the chart compares counts per category with the ground truth, and the table shows how many flows each flagged. In this example our model flags exactly the 258 attack flows.
- **Your own flows:** drop or choose a CSV with src_ip, dst_ip and all 83 trained features. The panel previews rows and columns before sending. A file missing any feature is **rejected** with the missing names listed, because a verdict computed on made-up zeros would look authoritative and mean nothing. Limits: 5 MB, 5,000 flows per request. For 50 flows or fewer a per-flow verdict table with confidences appears.

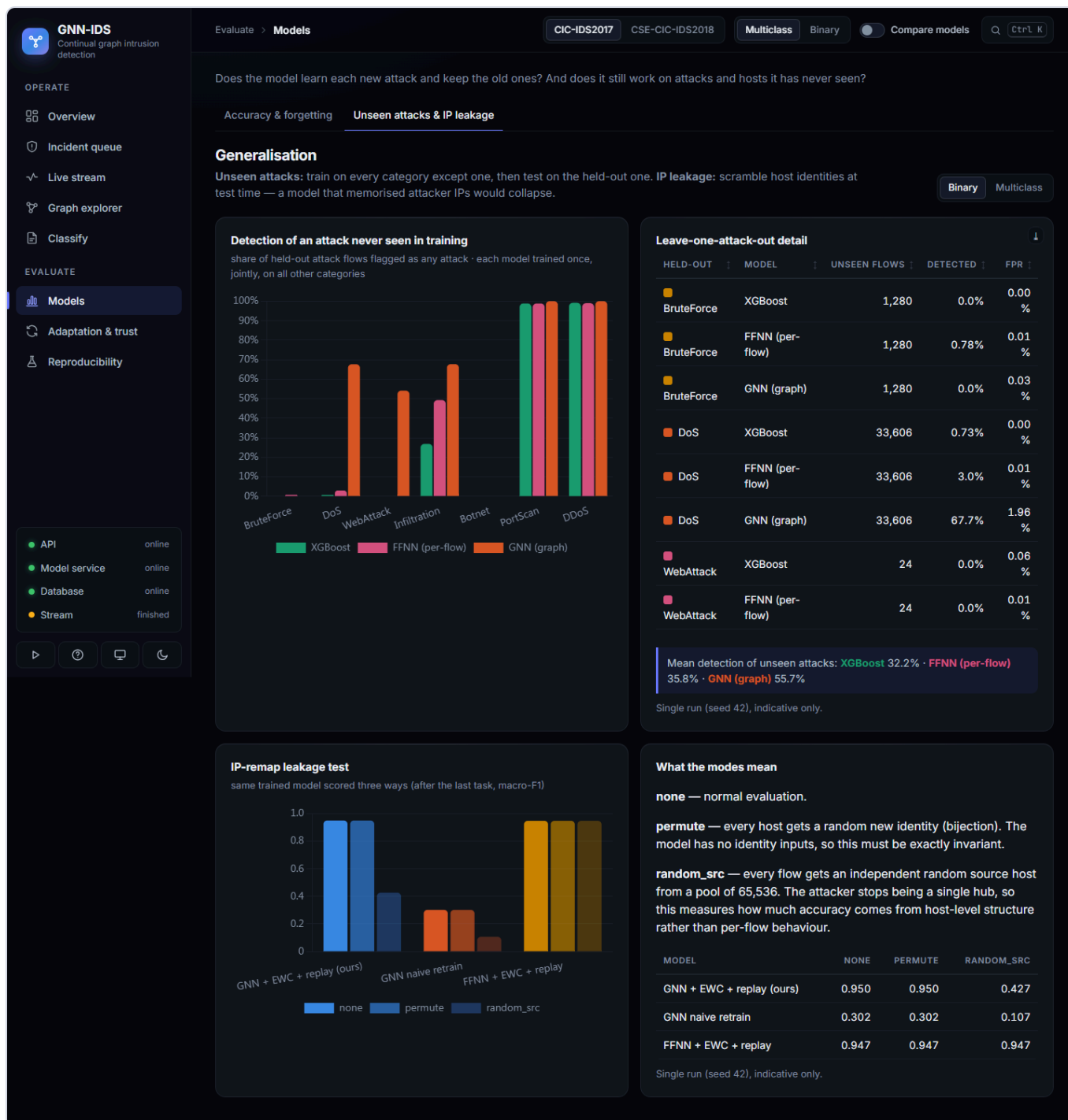
10.7 Models: the evidence about quality



- **Metric over tasks** (choose the metric): our model stays near the top; naive retraining and XGBoost fall as attacks are added.
- **Per-category recall matrix**: rows are "after task i", columns are attack types; forgetting shows as a column going dark.
- **Forgetting (BWT)** bar chart, and a **confusion matrix** for any model after any task.
- **Compare models** on adds every ablation (FFNN, replay-only, EWC-only, joint upper bounds).



With Compare models switched on.



Unseen attacks & IP leakage.

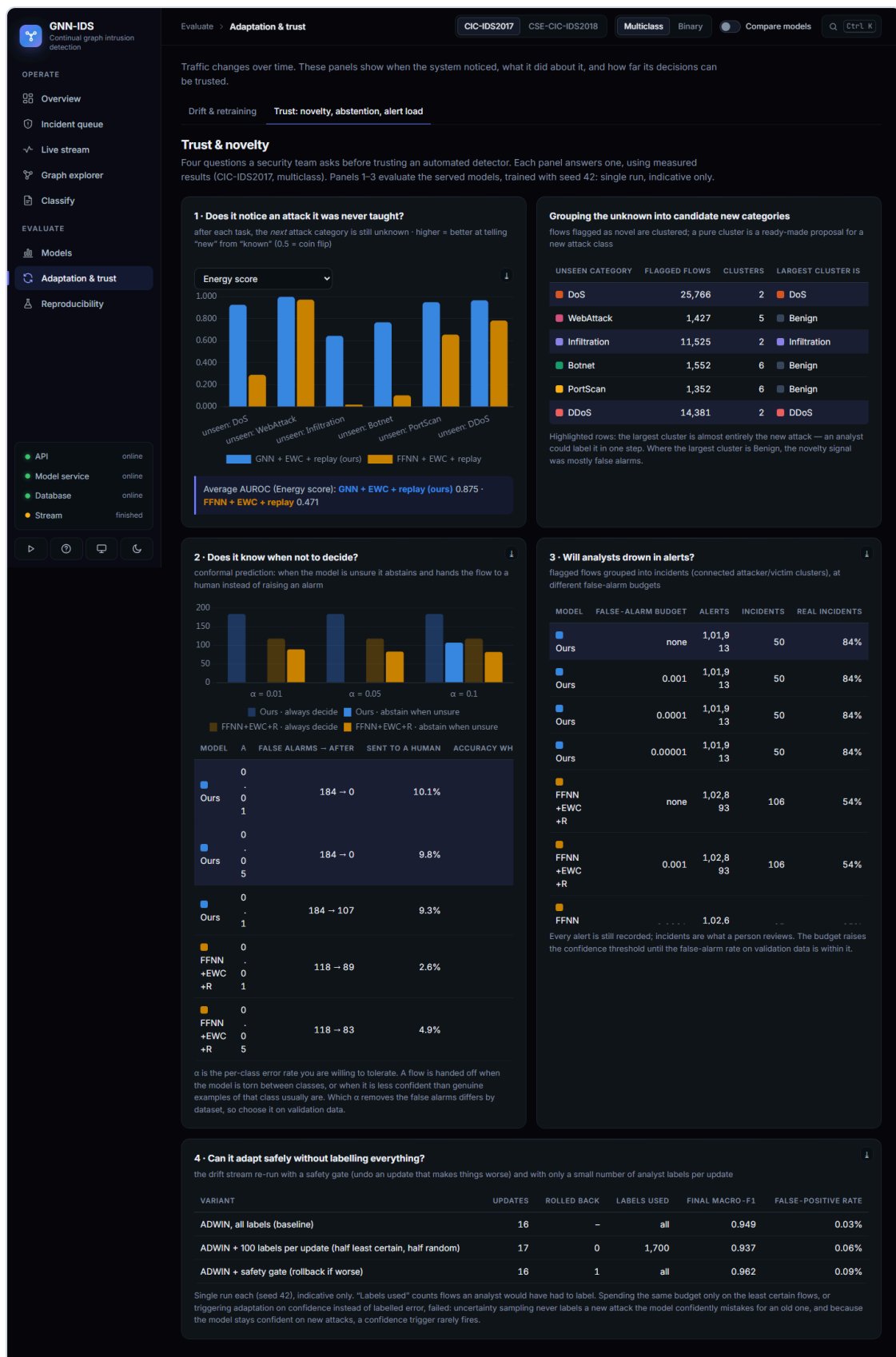
- **Unseen attacks:** each model trained without one category, then tested on it. Bars show the share detected. When a more than one seed exists the table adds **Seeds** and **Per seed** columns (CSE-CIC-IDS2018 BruteForce reads 99.7 % / 0.07 % / 100 % over three seeds; DoS 98.6 % on all three).
- **IP leakage test:** the same model scored normally, with host identities shuffled (*permute*: must not change, and does not), and with every flow's source randomised (*random_src*: the attacker stops being one hub; our model falls from 0.952 to 0.431 in all three seeds, the per-flow model is unaffected). This is the project's most important honest limitation.

10.8 Adaptation & trust



Drift & retraining.

- **Retrains vs final macro-F1:** bars are retrain cycles, the line is final quality, for ADWIN, a fixed schedule, the oracle and never adapting.
- **Policy scoreboard:** flags, retrains, macro-F1 and retention per model and policy, labelled with how many seeds are summarised.
- **Error timeline:** per-window error with drift flags and adaptations, for the selected run.



Trust: novelty, abstention, alert load.

1. Does it notice an attack it was never taught? AUROC per unseen category for three novelty scores (choose energy, max softmax or prototype distance), plus clustering of novel flows into candidate new categories.

2. **Does it know when not to decide?** Conformal abstention: false alarms before and after, share handed to an analyst.
3. **Will analysts drown in alerts?** Flow alerts vs incidents at different false-alarm budgets.
4. **Can it adapt safely without labelling everything?** All labels vs 100 labels per update (half least certain, half random) vs the safety gate that rolls back a bad update. Labelled single run, indicative only.

10.9 Reproducibility: check, don't trust

**GNN-IDS**
Continual graph intrusion detection

OPERATE

Overview

Incident queue

Live stream

Graph explorer

Classify

EVALUATE

Models

Adaptation & trust

Reproducibility

● APIonline

● Model serviceonline

● Databaseonline

● Streamfinished

▶

🕒

💬

🌙

CIC-IDS2017CSE-CIC-IDS2018MulticlassBinaryCompare modelsCtrl K

Reproducibility

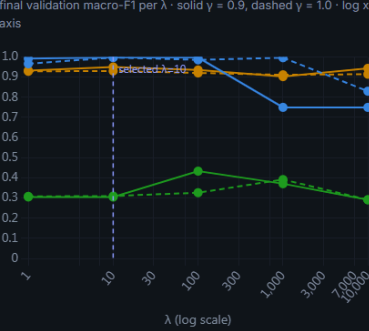
Every hyper-parameter was chosen on the **validation** split; test windows were only used by the final runs. Re-create every number with the command below.

Reproduce everything
data preparation → tuning → λ sweep → task sequence (3 seeds) → drift stream → unseen attacks → IP remap → figures → report

python -m experiments.reproduce_all --dataset cicids2017

Copy command

EWC λ sweep (validation)
final validation macro-F1 per λ · solid y = 0.9, dashed y = 1.0 · log x-axis



Selected per model: FFNN + EWC + replay λ=10, γ=0.9 · GNN + EWC only λ=100, γ=0.9 · GNN + EWC + replay (ours) λ=10, γ=0.9. Neighbouring λ values are often within run-to-run noise.

Training-budget tuning (validation)

MODEL	SETTING	VAL MACRO-F1	VAL FPR
Ours	train.gnn_epochs=3 replay.category_balanced_loss=false	0.641	0.01%
Ours	train.gnn_epochs=3 replay.category_balanced_loss=true	0.612	0.34%
Ours	train.gnn_epochs=10 replay.category_balanced_loss=false	0.996	0.00%
Ours	train.gnn_epochs=10 replay.category_balanced_loss=true	0.918	0.04%
FFNN+EWC+R	train.ffnn_epochs=3	0.872	0.05%
FFNN+EWC+R	train.ffnn_epochs=5	0.874	0.05%
FFNN+EWC+R	train.ffnn_epochs=10	0.920	0.04%
FFNN+EWC+R	train.ffnn_epochs=20	0.859	0.07%

Run metadata
results/cicids2017/multiclass/continual/run_info.json

timestamp (UTC)	2026-09-17T11:58:53.496641+00:00
seed (first run)	42
GPU	NVIDIA GeForce GTX 1650
python	3.12.10
torch	2.14.0+cu126
torch-geometric	2.8.0.post1
xgboost	3.4.1
scikit-learn	1.9.1
river	0.26.1
pandas	3.0.5
numpy	2.5.3
processes	
d-data hash	058f69000380
flows / windows / features	2099974 / 423 / 83
command	C:\Users\samar\Desktop\PBL\continual-learning-gnn-ids\experiments\run_continual.py --dataset cicids2017 --label-mode multiclass --seeds 42 43 44

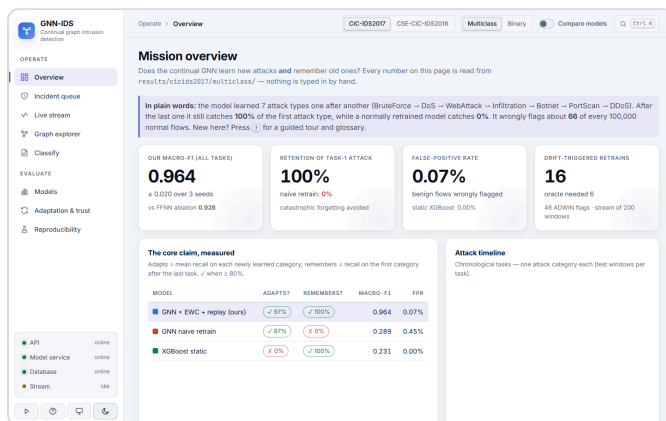
EWC stability (brief trap 3)
max lr·λ·max(F) reached per sweep run — above 1 the penalty step can overshoot

MODEL	Γ	Λ	MAX LR·Λ·F	VAL MACRO-F1
FFNN+EWC+R	0.9	1	0.002	0.932
FFNN+EWC+R	0.9	10	0.018	0.950
FFNN+EWC+R	0.9	100	0.254	0.935
FFNN+EWC+R	0.9	1000	2.436	0.902
FFNN+EWC+R	0.9	10000	33.360	0.943
FFNN+EWC+R	1	1	0.003	0.928
FFNN+EWC+R	1	10	0.017	0.930
FFNN+EWC+R	1	100	0.303	0.920
FFNN+EWC+R	1	1000	3.042	0.911
FFNN+EWC+R	1	10000	38.296	0.914
GNN EWC	0.9	1	0.002	0.305
GNN EWC	0.9	10	0.023	0.305

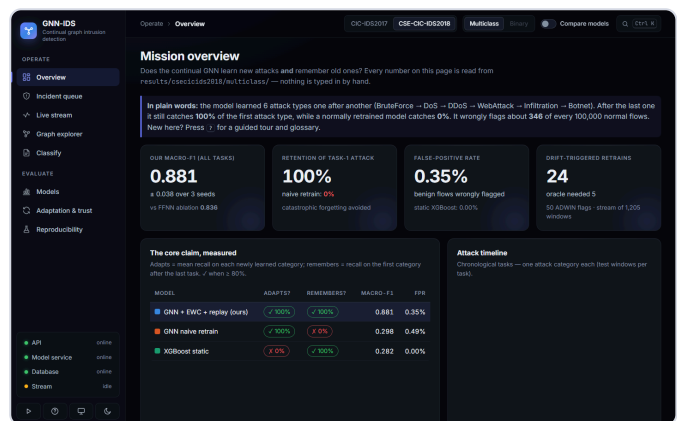
Reproducibility page.

- **Reproduce everything:** the single command (`python -m experiments.reproduce_all --dataset cicids2017`) with a Copy button.
- **EWC λ sweep:** validation macro-F1 for each penalty strength (log axis), the chosen value marked.
- **Training-budget tuning:** the settings tried on the validation split and the one selected.
- **Run metadata:** timestamp, seed, GPU, Python and package versions of the run that produced the numbers.
- **EWC stability:** a check that the penalty never became large enough to destabilise training.

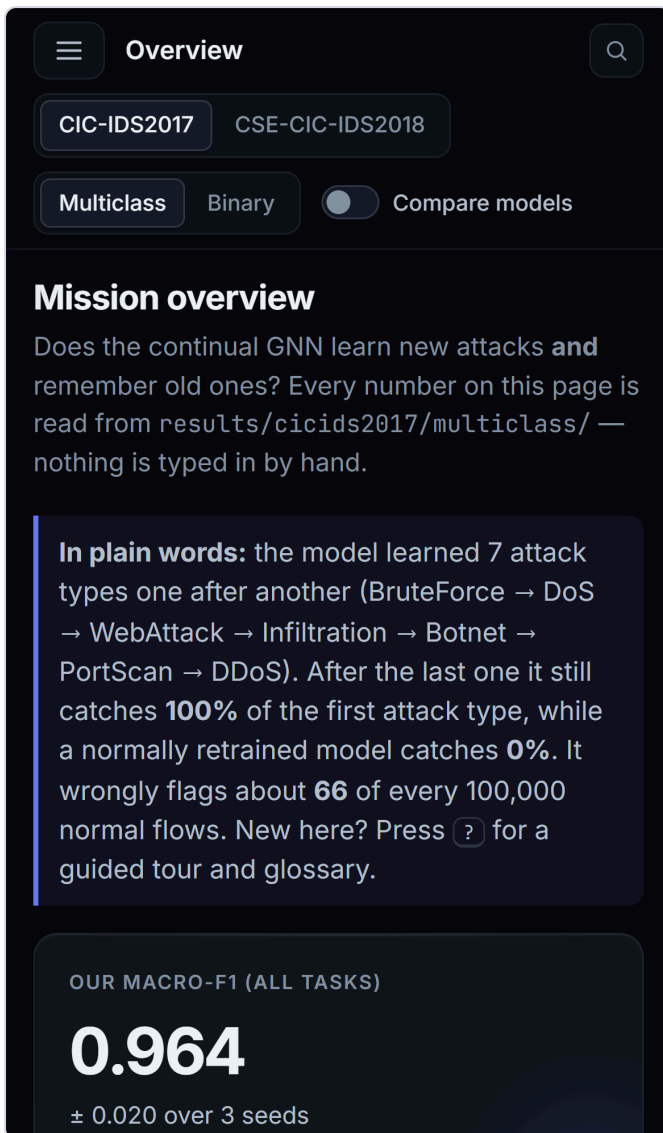
10.10 Light theme, the second dataset, and the phone



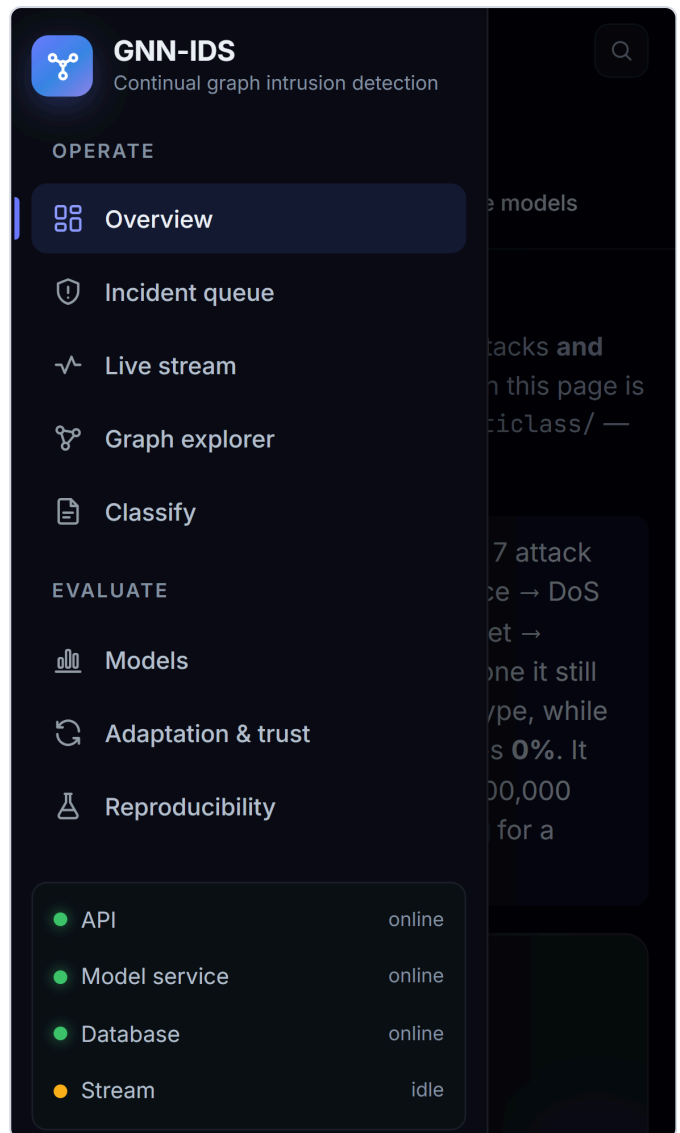
Light theme.



CSE-CIC-IDS2018 selected (Binary disabled).



Phone: page name next to the menu button.



Phone: the sidebar as a drawer.

11 · How it was built

11.1 Technology stack

Layer	Technology (version)	Why
Language	Python 3.12.10	The ML ecosystem.
Deep learning	PyTorch 2.14.0 (CUDA 12.6)	GPU training and inference.
Graphs	PyTorch Geometric 2.8.0	Graph data and message passing (paper 39). Neighbour sampling written by hand, so no fragile compiled add-ons are needed on Windows.
Tree baseline	XGBoost 3.4.1	The tree-ensemble comparison (paper 40).
Drift detection	river 0.26.1 (ADWIN)	Streaming change detection (paper 29).
Data	pandas 3.0.5, NumPy 2.5.3, PyArrow 25.0.1, scikit-learn 1.9.1, SciPy 1.18.1	Loading, scaling, metrics, statistics.
Plots for reports	matplotlib 3.11.2	Figures in results/.
API	FastAPI 0.141.1 + uvicorn 0.53.0, httpx 0.28.1	Two services (public API, model service), 47 routes.
Database	SQLAlchemy 2.0.54; SQLite by default, PostgreSQL/TimescaleDB optional	Flows, predictions, metrics, drift events, analyst decisions.
Console	Plain JavaScript ES modules (no framework, no build step), Chart.js 4.4.4, d3 7.9.0, self-hosted Inter and JetBrains Mono fonts	Nothing to compile; works offline; strict Content-Security-Policy.
Testing	pytest 9.1.1 (92 tests), Playwright 1.63.0 (browser smoke test), GitHub Actions	Every commit checked.
Hardware	Windows 11 laptop, NVIDIA GTX 1650 (4 GB), 8-core CPU, 24 GB RAM	Everything, including training, runs on one laptop.

11.2 Repository structure

src/ingestion, preprocessing	reading the CSVs, cleaning, scaling, task assignment
src/graph	windows → graphs, splits, neighbour sampling, IP remapping test
src/models	E-GraphSAGE, FFNN, EWC, replay buffers

<code>src/training, evaluation</code>	learners, task sequence, stream runner, metrics, conformal, calibration, open set
<code>src/drift</code>	ADWIN monitor
<code>src/product, explain</code>	incident grouping, response proposals, explanations
<code>src/api, db</code>	FastAPI services, database models
<code>experiments/</code>	one script per experiment; each writes CSV/JSON/PNG to results/
<code>dashboard/</code>	the console (about 2,650 lines of JavaScript)
<code>tests/</code>	92 pytest tests with small synthetic data
<code>docs/</code>	model card, positioning note, technical PDF, this guide, screenshots
<code>research paper/</code>	the 35 downloaded papers (kept out of git)

About 7,550 lines of Python in `src/` and `experiments/`; 75 commits from 17 September to 6 October 2026.

11.3 How it was built, step by step

1. Data pipeline on the corrected datasets, with tests on synthetic data.
2. The graph model, the per-flow model and the tree baseline, then EWC and replay, then the task-sequence experiment.
3. ADWIN drift detection and the streaming experiment.
4. Hyper-parameters tuned on the validation split only (epochs, class balancing, EWC λ and γ).
5. The API, database and console; then the product layer (incidents, explanations, abstention, novelty, dry-run response).
6. A results-integrity review that found and fixed several defects (for example a label-dependent sampling step that would have leaked labels into the graphs); affected experiments were re-run.
7. A senior review called the project "over-built and under-proven". The response: a temporal split, five seeds, paired statistics, a serving benchmark, calibration, a replay-budget sweep, a mixed-attack stream, extra seeds, removal of weak features (free-form CSV paste, a one-seed table, the Docker setup), the console redesign and a browser smoke test.

11.4 Is there an API key?

- **No external API keys are used anywhere.** No cloud service, no OpenAI/Claude or other language model, no paid API. Explanations are generated by rules in Python; everything runs locally.
- **The project's own API can require a key.** Set the environment variable `GNNIDS_API_KEY` before starting the stack. Then every endpoint except `/health` requires the header `X-API-Key`. The console asks for the key once, stores it in the browser and sends it only in that header (never in a web address). By default no key is set, which is fine on a single laptop and is listed as a limitation for any real deployment.

- **Other protections:** strict Content-Security-Policy on the console, checkpoints loaded in PyTorch's safe weights-only mode where possible, and response actions that are never executed.

11.5 How to run it

```
python -m venv .venv && .venv\Scripts\activate
pip install torch==2.14.0 --index-url https://download.pytorch.org/whl/cu126
pip install -r requirements.txt
python -m experiments.reproduce_all --dataset cicids2017      # all experiments
powershell -ExecutionPolicy Bypass -File scripts\run_stack.ps1 # start the console
# open http://localhost:8080 ; stop with scripts\run_stack.ps1 -Stop
python -m pytest                                           # 92 tests
python scripts\ui_smoke.py                                # browser smoke test
```

12 · What the results say (and do not say)

Experiment	Result	Seeds	Verdict
Learning 7 attacks in sequence, temporal split	Ours 0.915 [0.888, 0.935]; FFNN 0.870; naive 0.263	5	Works
Same, interleaved split	Ours 0.964; FFNN 0.928; naive 0.289	3	Works (optimistic split)
Does the graph help (seen attacks)?	+0.044 [+0.019, +0.058] temporal; +0.036 interleaved	5 / 3	Yes, modestly; indicative
Unseen attacks (CSE-CIC-IDS2018)	Graph >80 % in 6 of 12 tests; per-flow ≤ 0.07 % in all 12	2	Real, but which attacks varies by seed
Replay budget	0.304 / 0.702 / 0.952 / 0.948 / 0.957 at 0 / 1 / 5 / 10 / 20 windows	3	Replay is what stops forgetting
EWC on top of replay	No detectable gain interleaved; +0.067 temporal (two seeds)	3 / 5	Helps only on the harder split, indicatively
Drift: clean attack blocks	Fixed schedule 0.980 (8 retrains) beats ADWIN 0.960 (16.7)	3	ADWIN not efficient here
Drift: two attacks mixed	ADWIN 0.968 beats schedule 0.926	3	ADWIN more robust to mixing
Never adapting	0.230	3	Adaptation is essential
IP leakage	Permute: unchanged. Random sources: 0.952 $\rightarrow$ 0.431	3	No IP memorising; but depends on attacker topology
Incidents	~100,000 alerts $\rightarrow$ 50 incidents, 84 % real	1	Large workload reduction
Abstention (conformal)	184 GNN false alarms $\rightarrow$ 0, 9.8 % of flows to analyst ($\alpha = 0.05$)	1	Works on 2017; safe α differs per dataset
Calibration	ECE 0.52 % overall, 2.24 % on attacks (overconfident)	1	Overconfident on attacks
Speed	5.5 ms per 5,000-flow window on GPU; 10.5 ms on CPU	measured	Far faster than needed

"Indicative" means the bootstrap 95 % interval of the paired difference excludes zero but, with only three or five seeds, the Wilcoxon test cannot reach $p < 0.05$ however consistent the seeds are.

13 · Limitations and future work

Limitations (say these before an examiner does)

- **Topology dependence.** When attack traffic comes from many random sources the graph model collapses ($0.952 \rightarrow 0.431$, 3 seeds). Training with randomised sources recovers 0.914 but costs in-distribution accuracy (appendix in the README).
- **Lab data.** Both datasets are laboratory captures from 2017 and 2018 with attacks in clean blocks.
- **Few seeds.** Three to five seeds give indicative, not conclusive, statistics. Some product results are one run.
- **Labels are needed for drift.** ADWIN watches the labelled error; a label-free trigger missed most drift.
- **Seed instability.** On 2018 the identical configuration scored 0.855, 0.948 and 0.945 in three trainings; unseen BruteForce flipped between seeds.
- **Not a product.** No TLS, no roles, no rate limiting; flows stored in clear text; CSV input only.
- **Windows Smart App Control** blocked PyTorch for part of the project; it was turned off and the remaining experiments are being run (status in the README).

Future work

- Validate on newer and real traffic; ingest Zeek or Suricata output directly.
- Reduce topology dependence without losing accuracy (better augmentation, host-agnostic features).
- Learn from fewer labels; test label-free drift signals that work.
- Authentication, TLS and roles for a real deployment.

14 · Things you might have missed

14.1 Likely viva questions, with short answers

What is new here?	The combination, evaluated honestly: a graph detector that learns attacks continually, decides when to retrain, detects some never-seen attacks, and reduces analyst workload, with a temporal split, seeds and statistics. Not a new algorithm.
Why a graph instead of Random Forest?	Attacks have shapes between hosts that per-flow models cannot see; it detects never-seen attacks far better (>80 % in 6 of 12 tests vs ≤ 0.07 %), and it can learn new attacks without retraining on everything (section 7).
Your test windows are next to your training windows. Isn't that cheating?	That was the interleaved split. The temporal split tests on later traffic for every attack; it costs 0.049 macro-F1 and is reported alongside.
Is the difference statistically significant?	With 3–5 seeds no Wilcoxon test can reach $p < 0.05$; the bootstrap intervals exclude zero, so the gain is indicative. We say exactly that.
Does EWC help?	EWC alone fails. On top of replay: no detectable gain on the interleaved split; it prevented two collapses on the temporal split. Replay is the key ingredient.
Why 2017/2018 data?	Sequence of many attacks over days, IP addresses kept for graphs, independently corrected releases, comparability with published work, enterprise traffic (section 8).
Could it memorise IP addresses?	No: IPs are never features, and shuffling host identities changes nothing. But it does rely on attackers being few hosts.
Can it run in real time?	Scoring takes 5.5 ms per 5,000 flows on a laptop GPU; flow export and graph building from raw traffic are not in that figure.
What happens if the model is wrong?	It can abstain, every action is a dry run, and a safety gate can roll back a bad update.
Why does ADWIN retrain more than a schedule?	Attack bursts inside a task keep raising the error until the model has adapted. It wins when attacks are mixed, not when they arrive in clean blocks.
What would you do with more time?	Real traffic, less topology dependence, fewer labels, deployment security.

14.2 A ten-minute demo plan

1. Overview (1 min): the plain-words sentence, retention 100 % vs 0 %.
2. Incident queue (2 min): window #363, one incident, why it was flagged, the dry-run rule, approve.
3. Graph explorer (1 min): a scan or flood as a shape.
4. Live stream (2 min): start, point at a drift flag and the retrain.
5. Models (2 min): forgetting chart, then unseen attacks and the IP-leakage limitation.

6. Reproducibility (1 min): one command reproduces every number.

7. Close (1 min): what it is (a triage layer), what it is not (a SOC platform).

Do not start the Live stream while long experiments are training on the same 4 GB GPU.

14.3 Numbers worth memorising

0.915 / 0.964	our macro-F1, temporal (5 seeds) / interleaved (3 seeds)
100 % vs 0 %	retention: ours vs naive retraining
+0.044	graph vs per-flow on the temporal split, CI [+0.019, +0.058]
6 of 12 vs 0 of 12	never-seen attacks caught above 80 %: graph vs per-flow (2018)
0.304 → 0.952	replay-only with 0 vs 5 stored windows per attack type
100,000 → 50	alerts to incidents (84 % real)
5.5 ms	scoring a 5,000-flow window on the GPU
0.952 → 0.431	the topology limitation (random sources, 3 seeds)
69,128	model parameters; 83 features; 5,000 flows per window

14.4 Glossary

Flow	One network conversation summarised by 83 numbers.
Window	5,000 consecutive flows turned into one graph.
Task	A period in which a new attack type appears.
Macro-F1	Detection quality averaged over all categories (1.0 is perfect).
Retention	Recall on the first attack after learning all the others.
FPR	Share of normal traffic wrongly flagged.
BWT	Backward transfer: how much old tasks got worse; negative is forgetting.
Catastrophic forgetting	A network overwriting old knowledge when trained on new data.
EWC	Elastic Weight Consolidation: stiffen the weights that mattered before.
Replay	Rehearse a few stored old windows while learning new ones.
ADWIN	Adaptive windowing drift detector.
Concept drift	The data changing after training.
Conformal prediction	Prediction sets with coverage guarantees, used to abstain.
ECE	Expected calibration error: gap between confidence and accuracy.
AUROC	Area under the ROC curve; 0.5 is chance, 1.0 perfect separation.

Leave-one-attack-out	Train without one attack type, test on it (unseen-attack test).
Interleaved / temporal split	Test windows between training windows / strictly later in time.
Seed	The random starting point of a training run; more seeds = more trustworthy averages.
Dry run	An action that is proposed and recorded but never executed.
CEF	Common Event Format, a log line format that SIEM tools import.

14.5 Points you did not ask about but should know

- **Dataset licences:** both datasets are public for research; cite papers 01–03 when you use them. The downloaded papers in `research_paper/` are for your reading and are deliberately not pushed to GitHub.
- **Ethics:** the system never blocks traffic and never reads payloads; IP addresses are used only to build graphs.
- **Two deployment paths became one:** Docker was removed on 6 October 2026; `run_stack.ps1` is the only supported way to run it.
- **Where to find more:** README (all results), `docs/model-card.md`, `docs/positioning.md`, `DEMO.md`, and `docs/GNN-IDS_Technical_Documentation.pdf` (every API route and control).
- **One run outstanding:** the extra-seed runs (window size, drift, IP-remap and unseen attacks on both datasets, the topology re-run) are all in the README now; only the 5-seed re-run of the interleaved table is still to do, and it keeps three seeds. Nothing unmeasured is claimed.